WNDRVR
SUPPORT NETWORK

VXWORKS 7 FIREWALL PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Firewall Programmer's Guide, SR0650

16 September 2020

1. OVERVIEW

1. About Wind River Firewall

Wind River Firewall is based on a rule syntax compatible with *IP Filter*, the firewall filter package delivered with the NetBSD and FreeBSD operating systems. You can develop firewall rules using a simple keyword syntax and add those rules to the firewall with the Wind River Firewall application programming interface (API) or the ipf shell command.

About the Addresses Used in Examples

According to RFC 1918, the Internet Assigned Numbers Authority (IANA) has reserved the following three blocks of the IPv4 address space for private intermits:

- 10.0.0.0 - 10.255.255.255 (10/8 prefix)
- 172.16.0.0 - 172.31.255.255 (172.16/12 prefix)
- 192.168.0.0 - 192.168.255.255 (192.168/16 prefix)

These address spaces are also useful in networking examples, which need to function but also need to avoid public Internet addresses.

In this book, the 10/8 prefix, the largest of the three private address spaces, represents the public Internet. To represent a private address space, this book uses the 192.168/16 prefix.

2. Product Overview

Wind River Firewall provides the following features:

- IP filtering with stateful inspection for IPv4 or IPv6 packets.
- MAC (media access control) filtering.
- Logging at the network (L3) and data link (L2) layers.
- HTTP content filtering for URLs (both specific and by keyword), proxy traffic, Java applets, ActiveX controls, and cookies
- Nonvolatile (NV) storage of firewall rules.

IP filtering, MAC filtering, and logging are independent of each other. For example, you can install the IP filter, the MAC filter, or both. NV storage is implemented for both the IP and MAC filters.

General Purpose Features on page 2 Wind River Firewall includes the following features:
IP Filter Features on page 2 You can also write rules to filter IP packets based on the following conditions:
MAC Filter Features on page 3 You can also write rules to filter MAC frames based on the following conditions:
Filter Actions on page 3 You can apply the following actions to filtered packets:
Extensions on page 3 You can provide an extension handler and an HTTP handler.
HTTP Filtering on page 3 You can use HTTP content filtering to perform the following actions:
Sample Firewall Rules on page 4 Sample firewall rules for a typical home/SOHO (small office/home office) gateway are provided.
Management Features on page 4 Wind River Firewall provides the following management features:
Configuration Interfaces on page 4 Wind River Firewall provides the following configuration interfaces:
Network Address Translation (NAT) on page 4

2.1. General Purpose Features

Wind River Firewall includes the following features:

- An input filter.
- An output filter.
- Stateful inspection.
- Rate limiting.
- Filter on network interface.
- Rule grouping.

Stateful Packet Inspection

IP packet filtering alone cannot determine which packets are unsolicited and which packets are expected responses to legitimate requests. Stateful inspection determines which packets belong to legitimate connections. You can limit the number of concurrent stateful connections that Wind River Firewall allows to be open. You can combine stateful inspection with rate limiting, custom rules, or both.

Rate Limiting

Rate limiting lets you restrict the rate at which IP packets transit the firewall or their absolute quantity. You can also combine rate limiting with address filtering, limiting packets sent from a particular source address or going to a particular destination address. This feature helps defend against denial of service (DoS) flood attacks. You can also combine rate limiting with stateful inspection.

Firewall Support in Wind River Learning Bridge

Wind River Learning Bridge is a transparent layer 2 Ethernet learning bridge. For further information on its capabilities, see the *Wind River Learning Bridge for VxWorks 7 Programmer's Guide*. You can enable firewall support in the learning bridge. With this capability, you can:

- Filter MAC packets crossing the bridge.
- Filter stateful and stateless IP packets crossing the bridge.
- Filter logical link control/subnetwork access protocol (LLC/SNAP) frames in the learning bridge according to their DSAP or SSAP values or by SNAP type.

This functionality lets you create products with the capacity for complex filtering in multiple network topologies with multiple protocols. A device with this capability could be added to a network and begin filtering packets with only basic configuration, such as filtering rules.

Parent topic: Product Overview on page 1

2.2. IP Filter Features

You can also write rules to filter IP packets based on the following conditions:

- IPv4 addresses.
- IPv6 addresses.
- Protocol type.
- Fragments.
- Port numbers.

- TCP flags.
- ICMP type and code.
- Type of service or traffic class.
- IPv6 extension header.
- Time to live.
- Packet flags to indicate prior processing (IPsec, NAT, or tunneling).
- Connection states (keep state, keep state limited, keep ifstate, and keep ipsecstate).

Parent topic: Product Overview on page 1

2.3. MAC Filter Features

You can also write rules to filter MAC frames based on the following conditions:

- Network interface.
- Address.
- Frame type.
- Packet rate.

Parent topic: Product Overview on page 1

2.4. Filter Actions

You can apply the following actions to filtered packets:

- Reject the packet silently.
- Accept the packet silently.
- Log the packet. (See Logging on page 3, for further information.)
- Reject the packet and send back an ICMP message.
- Reject the packet and send back a TCP RESET packet.
- Perform the action specified by your custom extension handler, including rejecting packets from specific hosts and URLs.

Logging

Logging allows a packet that matches a filtering rule to be logged. Logged information is stored in memory and can be retrieved for display on the console.

Parent topic: Product Overview on page 1

2.5. Extensions

You can provide an extension handler and an HTTP handler.

Parent topic: Product Overview on page 1

2.6. HTTP Filtering

You can use HTTP content filtering to perform the following actions:

- Block access to websites based on specific URLs or URLs containing specific keywords.
- Block access to proxy servers that may circumvent the firewall's content filtering.
- Block Java applets, ActiveX controls, and cookies.

Parent topic: Product Overview on page 1

2.7. Sample Firewall Rules

Sample firewall rules for a typical home/SOHO (small office/home office) gateway are provided.

For more information, see Creating a Home/SOHO Gateway Firewall on page 14.

Parent topic: Product Overview on page 1

2.8. Management Features

Wind River Firewall provides the following management features:

- Logging to memory.
- Nonvolatile (NV) storage to file system.

NV Storage

The firewall can save and restore filter rules. NV storage is supported in the file system only.

Parent topic: Product Overview on page 1

2.9. Configuration Interfaces

Wind River Firewall provides the following configuration interfaces:

- Functions.
- Shell command.

API Library and Shell Command

The public API library contains utilities for IP filtering, MAC filtering, and logging. These functions are useful for testing and debugging. The ipf shell command provides access to the same functionality.

Note: Wind River Firewall does not support the use of virtual stacks.

Parent topic: Product Overview on page 1

2.10. Network Address Translation (NAT)

For information on Wind River NAT implementation, see *VxWorks 7 NAT Programmer's Guide*.

Parent topic: Product Overview on page 1

3. Additional Documentation

This Wind River Firewall manual focuses on configuring and using Wind River Firewall. Although the manual includes some general information about firewalls, it does not provide an exhaustive general discussion of firewall technology.

The following sections describe additional documentation about the technologies described in this book.

Wind River Documentation

The following Wind River documents present information that may be helpful when working with Wind River Firewall:

- VxWorks 7 Getting Started Guide
- VxWorks 7 Configuration and Build Guide
- VxWorks 7 NAT Programmer`s Guide
- Learning Bridge Programmer's Guide

Third-Party Documentation

- Stevens, W.R. *TCP/IP Illustrated, Volume 1: The Protocols*. ISBN-10: 0-201-63346-9. Reading, Massachusetts: Addison-Wesley, 1994.

  Cheswick, W.R. and Bellovin, *S.M. Firewalls and Internet Security: Repelling the Wily Hacker*. ISBN 0-201-63357-4. Reading, Massachusetts: Addison-Wesley, 1994.

  Zwicky, E.D., Cooper, S. and Chapman, D.B. *Building Internet Firewalls*, Second Edition. ISBN 1-56592-871-7. Sebastopol, California: O'Reilly and Associates, 2000.

4. RFCs

Wind River Firewall complies with the following RFCs, except where noted otherwise. These RFCs are accessible from the following website:

https://www.ietf.org/

- RFC 1918, *Address Allocation for Private Internets*. February 1996, Moskowitz, B., Karrenberg, D., de Groot, G. J., Lear, E.
- RFC 2196, *Site Security Handbook*. September 1997, Fraser, B.

2. CONFIGURING AND BUILDING WIND RIVER FIREWALL

1. Configuration and Build Workflow

Your VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries. You must also build a VxWorks run-time kernel image based on the kernel libraries.

For specific instructions on how to configure and build a VSB project and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the *VxWorks 7 Configuration and Build Guide*.

For a list of available source configuration options for the Wind River Firewall, see Source Build Configuration Options on page 6.

For a list of available kernel configuration components for Wind River Firewall, see Kernel Configuration Components (VIP) on page 7.

2. Source Build Configuration Options

Firewall VSB Options on page 6 table lists the required Wind River Firewall VSB configuration options.

Table 1. Firewall VSB Options

Workbench 4 Option Name	Description
IPNET_FIREWALL	Enables Wind River Firewall.

Adding Firewall Support in the Learning Bridge

To enable firewall support in the Wind River Learning Bridge, you must enable the following options in your VSB:

- **IPNET_BRIDGE**
- **IPFIREWALL_SUPPORT**

For more information, see the *Learning Bridge Programmer's Guide* .

Adding Firewall Support for a Bridge

To provide firewall support when using a bridge, add the following #define before the #ifdef **REMOVE_THIS_IFDEF_WHEN_SUPPORTING_IPBRIDGE** line in the *installDir* **\pkg_v2\ipnet-***x.x.x.x* **\coreip-***xxxx* **\src\ipcom\config\ipcom_pconfig.h** file:

```
#define REMOVE_THIS_IFDEF_WHEN_SUPPORTING_IPBRIDGE

#ifdef REMOVE_THIS_IFDEF_WHEN_SUPPORTING_IPBRIDGE
```

The #define enables the execution of the following code:

```
#ifdef _WRS_CONFIG_COMPONENT_IPFIREWALL_SUPPORT
#define INCLUDE_IPFIREWALL_SUPPORT
#define IPCOM_STORE_PHYSICAL_IF
#define IPFIREWALL_PHYSICAL_IF_SUPPORT
#endif

#ifdef _WRS_CONFIG_COMPONENT_BRIDGE_VLAN_SUPPORT
#define INCLUDE_IPBRIDGE_VLAN_SUPPORT
#endif

#ifdef _WRS_CONFIG_BRIDGE_STAT
#define IPBRIDGE_USE_STATS
#endif

#ifdef _WRS_CONFIG_BRIDGE_FILTERING_DATABASE_SIZE
#define IPBRIDGE_FILTERING_DB_SIZE _WRS_CONFIG_BRIDGE_FILTERING_DATABASE_SIZE
#endif

#ifdef _WRS_CONFIG_BRIDGE_MAX_PORTS
#define IPBRIDGE_MAX_PORTS _WRS_CONFIG_BRIDGE_MAX_PORTS
#endif
```

The #define **IPFIREWALL_PHYSICAL_IF_SUPPORT** line enables the phys option for the ipf block and ipf pass commands.

The phys option specifies a physical interface to be used in a filter rule when using a bridge.

3. Kernel Configuration Components (VIP)

The VIP component required to enable firewall functionality in your kernel image is INCLUDE_IPFIREWALL.

For information on optional kernel configuration components related to firewall, see Optional Kernel Configuration Components on page 8.

For information on setting the firewall configuration parameters, see Wind River Firewall Configuration Parameters on page 7.

4. Wind River Firewall Configuration Parameters

The following parameters are associated with the INCLUDE_IPFIREWALL component:

IPF_IPV4_RULE_FILE

Specifies the name of the default IPv4 IP filter rule file. The firewall will load the rules from this file if it exists at boot. Default is **fw4.cfg**.

IPF_IPV6_RULE_FILE

Specifies the name of the default IPv6 IP filter rule file. The firewall will load the rules from this file if it exists at boot. Default is **fw6.cfg**.

IPF_FWMAC_RULE_FILE

Specifies the name of the default MAC filter rule file. The firewall will load the rules from this file if it exists at boot. Default is **fwmac.cfg**.

IPF_ICMP_TIMEOUT

Specifies the timeout until an ICMP stateful mapping times out. Default is 60 seconds.

IPF_MAX_STATEFUL_MAPPINGS

Specifies the maximum number of stateful mappings the firewall is able to handle. Default is 1,000.

IPF_OTHER_TIMEOUT

Specifies the timeout until other stateful mappings time out. Default is 60 seconds.

IPF_TCP_TIMEOUT

Specifies the timeout until a TCP stateful mapping times out. Default is 432,000 seconds (5 days).

IPF_UDP_TIMEOUT

Specifies the timeout until a UDP stateful mapping times out. Default is 60 seconds.

IPF_MAX_IP_RULES

Specifies the maximum number of IP rules you can configure for the firewall. Default is 100.

IPF_MAX_MAC_RULES

Specifies the maximum number of MAC rules you can configure for the firewall. Default is 100.

IPF_MAX_RULE_ARGS

Specifies the maximum number of arguments that can be included in a firewall rule string. Default is 16.

5. Optional Kernel Configuration Components

To access the firewall shell commands, include the INCLUDE_IPFIREWALL_CMD component.

6. Wind River Firewall and Symmetric Multiprocessing

If you build Wind River Firewall for use on a target configured with symmetric multiprocessing (SMP), the SMP capability of firewall is automatically enabled. The firewall hooks will run in parallel on multiple cores, resulting in improved performance.

7. Adding a Hook for Firewall Rules

If you plan to add firewall rules at startup by calling the ipfirewall_add_rule( ) function, add a hook for those rules. To create this hook, add a **USER_APPL_INIT** macro in the BSP in the **usrAppinit.c** file located in your VIP project directory.

The macro is similar to the following:

```
#define INCLUDE_USER_APPL
#define USER_APPL_INIT \
{ \
IMPORT void usrFwAddRules();\
usrFwAddRules();\
}
```

In this example, usrFwAddRules( ) is a sample function only. It is not distributed with your product installation. You must create it (or a function with a similar name) yourself.

Note: Some BSPs include sample definitions of **INCLUDE_USER_APPL** and **USER_APPL_INIT**. If so, remove those examples. Define **INCLUDE_USER_APPL** and **USER_APPL_INIT** only once.

3. TUTORIAL

1. Introduction

This chapter contains tutorials that guide you through the creation of two projects:

- Creating and building a simple firewall.
- Creating and building a typical firewall for a home/SOHO gateway protecting a private network.

Both tutorials provide information on writing firewall rules and testing the firewall. Both tutorials also provide information on using Workbench 4 to develop and deploy your firewall.

2. Network Configuration

The firewalls created in these tutorials are both designed to run on a simple network consisting of the following nodes:

- A public host.
- Two private hosts.
- A gateway with two interfaces.
- Switches (optional).

Tutorial Network--Nodes and Software Requirements table on page 10 provides configuration information for each node.

Table 1. Tutorial Network--Nodes and Software Requirements

Node	IP Address	Required Software
A (public host)	10.31.100.21	**Web server** **Web browser** **Ping command**
B (private host)	192.168.74.2	**Web server** **Web browser** **Ping command** **Telnet client command**
C (private host)	192.168.74.3	**Web server** **Web browser** **Ping command** **Telnet client command (optional)**

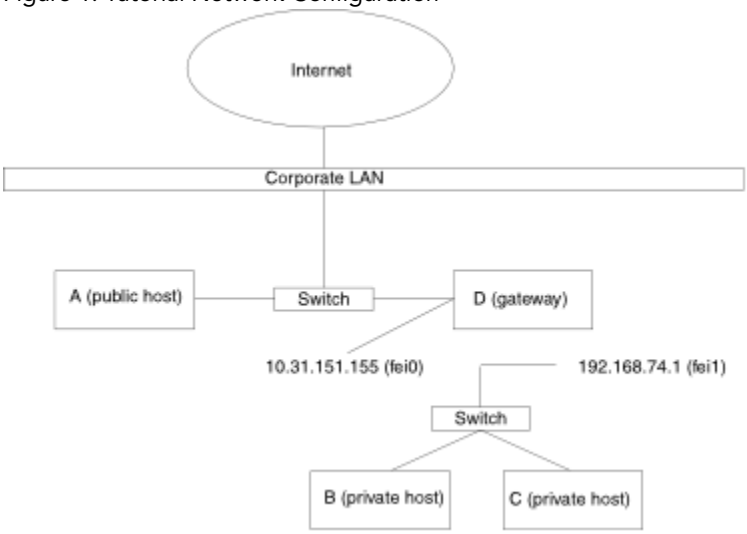
Node	IP Address	Required Software
D (gateway)	**10.31.151.155 on fei0 (public interface)** **192.168.74.1 on fei1 (private interface)**	

Note: Host A must be configured with a route to the 192.168.74.0/24 network through 10.31.151.155.

Hosts B and C must be configured with a route to the 10.31.0.0/16 network through 192.168.74.1.

If desired, this network can also be connected to a corporate LAN and, through that LAN, to the Internet. Tutorial Network Configuration on page 11 illustrates this configuration.

Figure 1. Tutorial Network Configuration



3. Creating a Simple Firewall

This tutorial explains how to create a simple firewall using basic IP filtering rules.

Detailed information on the IP filter is provided in Creating an IP Filter on page 34.

Note: The steps in the following sections assume you have built and configured your VxWorks image. For configuration and build instructions, see the *VxWorks 7 Configuration and Build Guide*. For configuration information specific to firewall, see Configuring and Building Wind River Firewall on page 6.

Security Policy on page 11 The security policy implemented in this tutorial is designed to do the following:
Writing Rules on page 12
Testing the Firewall on page 13 Test the firewall to verify that it is accepting and rejecting traffic as expected.

3.1. Security Policy

The security policy implemented in this tutorial is designed to do the following:

- Block all packets from the public network going to the private network unless private hosts have requested the incoming traffic.
- Pass and log incoming ICMP echo requests to host C.
- Pass outbound TCP/UDP packets from any private network host and keep state. Inbound packets arriving in response to such requests are allowed to pass the firewall.
- Pass outbound ICMP packets from any private network host and keep state. Inbound packets arriving in response to such requests are allowed to pass the firewall.

This security policy uses stateful packet inspection. Stateful inspection allows outgoing connections to be established, but does not allow uninitiated connections from the public network. For more information on stateful packet inspection, see Stateful Inspection on page 38.

Note: This tutorial assumes that you have already connected the required hardware, created a VxWorks source build project and a VxWorks image project, and added a hook for firewall rules (Adding a Hook for Firewall Rules on page 8). If you have not already performed these tasks, do so now. For further information, see Network Configuration on page 10.

Parent topic: Creating a Simple Firewall on page 11

3.2. Writing Rules

About This Task

This section describes how to develop the rules to fulfill the security policy described in Security Policy on page 11. All rules should be added to the **usrAppInit.c** file in your VxWorks image project.

For information on firewall keywords and parameters, see Keywords Reference on page 54.

Procedure

1. Create a default rule. Create a rule that blocks all incoming packets unless other rules explicitly allow the packets to pass. To create a rule, call the ipfirewall_add_rule( ) function, using the appropriate keywords as parameters. For a simple firewall, the function is as follows:

```
ipfirewall_add_rule( AF_INET, "block in on fei0 all" );
```

2. Create a rule to accept incoming ICMP echo requests. Create a rule that accepts incoming ICMP echo requests to private host C and logs these packets. Use the following function:

```
ipfirewall_add_rule( AF_INET, "pass in log quick on fei0 proto icmp from any
to 192.168.74.3 icmp-type 8" )
```

   Note the use of the **quick** keyword in these functions. This keyword instructs the firewall to abort processing on the first match and immediately take the action specified in the rule. For further information, see *Controlling Rule Processing on page 23*.

3. Create a rule to keep state on outbound TCP/UDP packets. Create a rule that passes outbound TCP/UDP packets from any private network host and records their state. With this rule, the firewall automatically passes inbound packets arriving in response to such requests. Use the following function:

```
ipfirewall_add_rule( AF_INET, "pass out quick on fei0 proto tcp/udp from
192.168.74.0/24 to any keep state" )
```

4. Create a rule to keep state on outbound ICMP packets. Create a rule that passes outbound ICMP packets from any private network host and records their state. With this rule, the firewall automatically passes inbound packets arriving in response to such requests. This rule is nearly identical to the previous one. Use the following function:

```
ipfirewall_add_rule( AF_INET, "pass out quick on fei0 proto icmp from
192.168.74.0/24 to any keep state" )
```

Results

Complete Code—Simple Firewall

When complete, the firewall code should look something like the following:

```
#define USER_APPL_INIT_FIREWALL { \
    IMPORT int ipfirewall_add_rule(int family, const char *rule); \
    /* Default rule to block all traffic */ \
    ipfirewall_add_rule( AF_INET, "block in on fei0 all" );
    /* Pass in and log ICMP packets */ \
    ipfirewall_add_rule( AF_INET, "pass in log quick on fei0 proto icmp from any
    to 192.168.74.3 icmp-type 8" )
    /* Pass outbound TCP\UDP packets and keep state */ \
    ipfirewall_add_rule( AF_INET, "pass out quick on fei0 proto tcp/udp from
    192.168.74.0/24 to any keep state" )
    /* Pass outbound ICMP packets and keep state */ \
    ipfirewall_add_rule( AF_INET, "pass out quick on fei0 proto icmp from
    192.168.74.0/24 to any keep state" )
}
/*****************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
{
#ifdef     USER_APPL_INIT
           USER_APPL_INIT; /* for backwards compatibility */
#endif
                           /* add your application code here */
    ...
}
```

Parent topic: Creating a Simple Firewall on page 11

3.3. Testing the Firewall

Test the firewall to verify that it is accepting and rejecting traffic as expected.

1. Perform the following tests:
   - Web browsing from B to A.
   - Web browsing from C to A.
   - Ping from B to A.
   - Ping from C to A.
   - Ping from A to C.

WNDRVR

These tests should all pass.

2. Next, perform the following tests:
   - Web browsing from A to B.
   - Web browsing from A to C.
   - Ping from A to B.

   These tests should all fail.
3. Check the firewall log by issuing the following shell command:

```
-> cmd
[vxWorks *]# ipf -Pl
```

   Verify that the ping from A to C was logged.
4. Check firewall statistics by issuing the following shell command:

```
[vxWorks *]# ipf -S
```

   Counters for states added, states expired, states expired, logged input packets, blocked input, and passed input should all be greater than 0. The firewall statistics are similar to the following:

```
FIREWALL STATISTICS:
 input packets: blocked 23 passed 21 nomatch 0
 output packets: blocked 0 passed 32 nomatch 0
 invalid packets: 0
 logged input packets: blocked 0 passed 16
 logged output packets: blocked 0 passed 0
 log failures: 0
 states added: 17
 states expired: 17
 state hits: 0
 state failures: 0
 input mac frames: blocked 0 passed 0 nomatch 0
 output mac frames: blocked 0 passed 0 nomatch 0
 invalid mac frames: 0
 logged input mac frames: blocked 0 passed 0
 logged output mac frames: blocked 0 passed 0
 mac log failures: 0
```

Parent topic: Creating a Simple Firewall on page 11

4. Creating a Home/SOHO Gateway Firewall

The preceding tutorial illustrated how to create a basic firewall. This tutorial presents a more complex example that can serve as the basis for a firewall running on a home/SOHO gateway. The network configuration is the same one used in the preceding tutorial. For more information, see Network Configuration on page 10.

Note: No services are available on any internal host.

Security Policy on page 15 The security policy for the home/SOHO gateway is to:
Writing Rules on page 15

Testing the Firewall on page 17 Test the firewall to verify that it is working and accepting and rejecting traffic as expected. The home/SOHO gateway firewall should pass and fail the same tests as the simple tutorial.

4.1. Security Policy

The security policy for the home/SOHO gateway is to:

- Block all incoming traffic from the Internet unless a private host has connected to a server on the Internet.
- Block all incoming traffic from private networks.
- Block incoming traffic from special networks, such as those used for internal communication, automated system configuration, and similar purposes.
- Block multicast traffic.
- Block and log spoofing and smurf relay attacks.
- Pass outbound TCP/UDP packets from any private network host and keep state. Inbound packets arriving in response to such requests are allowed to pass the firewall.
- Pass outbound ICMP packets from any private network host and keep state. Inbound packets arriving in response to such requests are allowed to pass the firewall.

Parent topic: Creating a Home/SOHO Gateway Firewall on page 14

4.2. Writing Rules

About This Task

This section describes how to develop the rules to fulfill the security policy described in Security Policy on page 15. All rules should be added to the **usrAppInit.c** file in your VxWorks image project.

Procedure

1. Create a default rule. Create a rule that blocks all incoming packets unless other rules explicitly allow the packets to pass. As in the previous tutorial, call ipfirewall_add_rule( ), using the appropriate keywords as parameters. For the SOHO gateway firewall, the function is as follows:

```
ipfirewall_add_rule(AF_INET, "block in on fei0 all"); \
```

2. Block all private networks. Create rules to block traffic from private networks. Use the following functions:

```
ipfirewall_add_rule(AF_INET, "block in quick on fei1 from 192.168.0.0/16 to
any"); \
ipfirewall_add_rule(AF_INET, "block in quick on fei1 from 172.16.0.0/12 to
any"); \
ipfirewall_add_rule(AF_INET, "block in quick on fei1 from 10.0.0.0/8 to
any"); \
```

   Note the use of the **quick** keyword in these functions. This keyword instructs the firewall to abort processing on the first match and immediately take the action specified in the rule. For further information, see *Controlling Rule Processing on page 23*.

3. Block all special networks. Create rules to block traffic from special networks. Use the following functions:

```
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 127.0.0.0/8 to any"); \
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 0.0.0.0/8 to any"); \
```

```
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 169.254.0.0/16 to any"); \
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 192.0.2.0/24 to any"); \
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 204.152.64.0/23 to any"); \
```

4. Block multicast traffic. Create a rule to block all multicast traffic. Use the following function:

```
ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 224.0.0.0/3 to any"); \
```

5. Block and log spoofing attacks. Create a rule to block and log spoofing attacks. Use the following function:

```
ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from 192.168.74.0/24 to any"); \
```

6. Block and log smurfing attacks. Create a rule to block and log possible smurf attacks. Use the following functions:

```
ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from any to 192.168.74.0/32"); \
   ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from any to 192.168.74.255/32"); \
```

7. Create a rule to keep state on outbound TCP/UDP packets. Create a rule that passes outbound TCP/UDP packets from any private network host and records their state. With this rule, the firewall automatically passes inbound packets arriving in response to such requests. Use the following function:

```
ipfirewall_add_rule(AF_INET, "pass out quick on fei0 proto tcp/udp from 192.168.74.0/24 to any
keep state"); \
```

8. Create a rule to keep state on outbound ICMP packets. Create a rule that passes outbound ICMP packets from any private network host and records their state. With this rule, the firewall automatically passes inbound packets arriving in response to such requests. This rule is nearly identical to the previous one. Use the following function:

```
ipfirewall_add_rule(AF_INET, "pass out quick on fei0 proto icmp from 192.168.74.0/24 to any kee
p state"); \
```

Results

Complete Code—Home/SOHO Gateway Firewall

When complete, the firewall code is similar to the following:

```
#include <vxWorks.h>
#if defined(PRJ_BUILD)
#include "prjParams.h"
#endif /* defined PRJ_BUILD */

#ifndef AF_INET
#define AF_INET 2
#endif

#ifndef INCLUDE_USER_APPL
#define INCLUDE_USER_APPL
#endif

#define USER_APPL_INIT_FIREWALL { \
    IMPORT int ipfirewall_add_rule(int family, const char *rule); \
    /* Default rule to block all traffic */ \
    ipfirewall_add_rule(AF_INET, "block in on fei0 all"); \
    /* Block private networks */ \
```

```
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 192.168.0.0/16 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 172.16.0.0/12 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 10.0.0.0/8 to any"); \
    /* Block special networks */ \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 127.0.0.0/8 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 0.0.0.0/8 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 169.254.0.0/16 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 192.0.2.0/24 to any"); \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 204.152.64.0/23 to any"); \
    /* Block multicast */ \
    ipfirewall_add_rule(AF_INET, "block in quick on fei0 from 224.0.0.0/3 to any"); \
    /* Block and log possible spoofing attacks */ \
    ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from 192.168.74.0/24 to any"); \
    /* Block and log possible smurf attacks */ \
    ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from any to 192.168.74.0/32"); \
    ipfirewall_add_rule(AF_INET, "block in log quick on fei0 from any to 192.168.74.255/32"); \
    /* Enable stateful firewall on outgoing traffic */ \
    ipfirewall_add_rule(AF_INET, "pass out quick on fei0 proto tcp/udp from 192.168.74.0/24 to an
y keep state"); \
    ipfirewall_add_rule(AF_INET, "pass out quick on fei0 proto icmp from 192.168.74.0/24 to any ke
ep state"); \
}
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
{
#ifdef     USER_APPL_INIT
           USER_APPL_INIT;          /* for backwards compatibility */
#endif
```

Parent topic: Creating a Home/SOHO Gateway Firewall on page 14

4.3. Testing the Firewall

Test the firewall to verify that it is working and accepting and rejecting traffic as expected. The home/SOHO gateway firewall should pass and fail the same tests as the simple tutorial.

1. Perform the following tests:
   - Web browsing from B to A.
   - Web browsing from C to A.
   - Ping from B to A.
   - Ping from C to A.
   - Ping from A to C.

   These tests should all pass.
2. Next, perform the following tests:
   - Web browsing from A to B.
   - Web browsing from A to C.
   - Ping from A to B.

   These tests should all fail.
3. Check the firewall log by issuing the following shell command:

```
[vxWorks *]# ipf -Pl
```

Verify that the ping from A to C was logged.

> **Note:** To run this command, you must switch to the command interpreter shell before running the ipf command. Type cmd at the command prompt. Then run the ipf command.

4. Check firewall statistics by issuing the following shell command:

```
[vxWorks *]# ipf -S
```

Counters for states added, states expired, logged input packets, blocked input, and passed input should all be greater than 0. The firewall statistics are similar to the following:

```
FIREWALL STATISTICS:
input packets: blocked 23 passed 16 nomatch 0
output packets: blocked 0 passed 0 nomatch 0
invalid packets: 0
logged input packets: blocked 23 passed 16
logged output packets: blocked 0 passed 0
log failures: 0
states added: 17
states expired: 17
state hits: 0
state failures: 0
input mac frames: blocked 0 passed 0 nomatch 0
output mac frames: blocked 0 passed 0 nomatch 0
invalid mac frames: 0
logged input mac frames: blocked 0 passed 0
logged output mac frames: blocked 0 passed 0
mac log failures: 0
```

> **Note:** The exact statistics you see vary according to the test parameters actually used.

These tests verify the proper functioning of the stateful firewall rules described in steps *7 on page 16* and *8 on page 16*. These tests do not verify functioning of the rules that block special networks and spoofing/smurf attacks. Testing these additional rules requires more computers and the ability to send packets with a faked source IP address. These test procedures are outside the scope of this tutorial.

Parent topic: Creating a Home/SOHO Gateway Firewall on page 14

4. FUNDAMENTALS

1. Introduction

This chapter describes basic firewall concepts, including the elements of a firewall rule, the processing of rules and rule groups, different methods for writing rules, and rate limiting.

The following chapters provide additional information:

- Creating an IP Filter on page 34 for a description of IP-specific filtering methods
- Creating a MAC Filter on page 42 for a description of MAC-specific filtering methods
- Filtering HTTP Content on page 50 for a description of methods for filtering HTTP traffic

Note:

For security concerns, the **IPFIREWALL_BLOCK_IP_INVALID** macro is defined by default in **ipfirewall_config.h**, so that broken packages are dropped, rather than passed on. You can change this behavior by editing **ipfirewall_config.h** to comment out the define. This file is located in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/firewall-**x.x.x.x **/h**

2. Firewall Operation

A firewall is a collection of rules for inspecting and filtering data packets and frames as they enter, transit, and exit the TCP/IP stack. Wind River Firewall provides two filters:

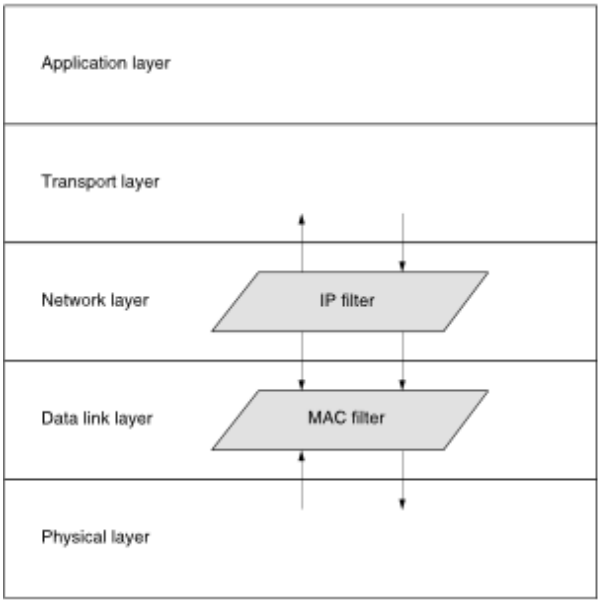
- An IP filter, which operates in the network layer.
- AMAC filter, which operates in the data link layer.

At the network layer, the firewall checks every incoming IP packet against the rules in the IP filter. If there is a matching rule, the firewall either blocks the packet or passes it to the transport layer, based on the action specified in the rule. The firewall also checks every outgoing packet against the rules in the IP filter. If there is a matching rule, the firewall either blocks the packet or passes it to the data link layer, based on the action specified in the rule.

At the data link layer, the firewall checks every incoming Ethernet frame against the rules in the MAC filter. If there is a matching rule, the firewall silently blocks the frame or passes it to the network layer, based on the action specified in the rule. The firewall also checks every outgoing frame against the rules in the MAC filter and either drops the frame or passes it to the physical layer for output.

Wind River Firewall Schematic on page 19 illustrates the flow of data through the network and data link layers of the TCP/IP stack.

Figure 2. Wind River Firewall Schematic



Note: Filtering rules implement your security policy. Before you start writing filtering rules, develop your security policy carefully and in detail.

3. Elements of a Firewall Rule

Each firewall rule consists of at least the following elements:

- An action to be taken (either **block** or **pass**).
- The direction, or filter location (either **in** or **out**).
- An address scope, such as **all**, **me**, or **any**.

Additional optional parameters can also be specified.

Action to Be Taken on page 20 At a minimum, each rule must specify whether a packet is to be blocked or passed, using the keyword **block** or **pass**. For example:
Address Scope on page 21
Interface on page 21 Use the **on** keyword to narrow the scope of a rule to packets being sent or received on a particular interface. For example:

3.1. Action to Be Taken

At a minimum, each rule must specify whether a packet is to be blocked or passed, using the keyword **block** or **pass**. For example:

```
pass out all
```

This rule instructs the firewall to allow all packets to exit the system.

Additional actions are also possible. For example:

```
block in log all
```

This rule instructs the firewall to block all incoming packets and log them for later examination. Logging is available for both IP and MAC filters. For more information, see *Logging on page 25*.

For IP filter rules, you can also instruct the firewall to notify the peer when a packet is dropped. For further information, see *Responding to Blocked Packets on page 40*.

Parent topic: Elements of a Firewall Rule on page 20

3.2. Address Scope

For further information on defining the address scope of a rule, see *IP Filter Address Scope on page 35*, and *Filtering by Address*.

Parent topic: Elements of a Firewall Rule on page 20

3.3. Interface

Use the **on** keyword to narrow the scope of a rule to packets being sent or received on a particular interface. For example:

```
block in on ppp0 all
pass out on fei0 all
```

You can also use the plus sign (**+**) to specify more than one interface. For example:

```
pass out on fei+ all
```

This rule instructs the firewall to pass all packets on any interface containing the characters *fei*—fei0, fei1, fei2, and so forth.

Note: Other methods of filtering are also available. For further information on IP filter rules, see *Creating an IP Filter*. For further information on MAC filter rules, see *Creating a MAC Filter on page 42*.

Parent topic: Elements of a Firewall Rule on page 20

4. Methods for Writing Rules

You can develop firewall rules using a simple keyword syntax and add these rules to the firewall with a rule file, the Wind River Firewall API, or the ipf shell command.

Using a Rule File on page 21
Using the API on page 22
Using a Shell Command on page 22

4.1. Using a Rule File

You can write firewall rules using the keyword syntax shown in the examples provided in *Elements of a Firewall Rule on page 20*. Save your firewall rules in a text file and store them wherever you like on the system. You can use any file name or extension, but it is common to use a **.cfg** extension. The file must reside on the target and be stored on local media.

You should maintain separate rule files for IPv4, IPv6, and MAC rules. Certain rules, such as **block in all**, have the same syntax for each filter type. Segregating them in separate keyword files allows the firewall to apply the rules correctly.

Empty lines and white space are permitted in a rule file. The pound sign (**#**) precedes a comment. You can terminate a line with a comment.

The following example shows a comment line, an empty line, a line with a rule terminated by a comment, and a line with a rule. The entire file consists of four lines.

```
# example rule file

block in on fei0 all # default action is to block all incoming packets
pass out quick on fei0 proto tcp/udp from any to any keep state
```

For further information on a particular keyword, see its reference entry in *Keywords Reference on page 54*.

Parent topic: Methods for Writing Rules on page 21

4.2. Using the API

You can also use the Wind River Firewall API to create firewall rules. All rules should be added to the **usrAppInit.c** file in your VxWorks image project. For a description of the syntax and available parameters, see the reference entry for each function.

Parent topic: Methods for Writing Rules on page 21

4.3. Using a Shell Command

You can also use the ipf shell command to create firewall rules. For a description of all available parameters, see *Shell Command Reference on page 73*.

Parent topic: Methods for Writing Rules on page 21

5. Rules and Rule Groups

Arranging rules in groups can improve performance for complex rule sets. Grouping allows you to arrange rules in a treelike structure instead of a linear list.

The advantage to rule grouping is that if a packet fails to match the head rule in a group, the firewall skips the remaining subrules and immediately begins matching against the next group. This feature is available for both IP and MAC filters.

When you add rules to a firewall without specifying a rule group, the new rules are added to the default group. To create a group, use the **head** keyword, followed by a group number. The number 0 is reserved for the default group, therefore use a number greater than 0. Each group must have a head rule, followed by any number of subrules.

To assign a subrule to a group, use the **group** keyword, followed by the group number. For example:

```
block in quick on fei0 all head 1
   block in quick on fei0 from 10.0.0.0/8 to any group 1
   block in quick on fei0 from 11.0.0.0/8 to any group 1
   pass in on fei0 all group 1
block in quick on fei1 all head 2
   block in quick on fei1 from 10.0.0.0/8 to any group 2
   block in quick on fei1 from 11.0.0.0/8 to any group 2
   pass in on fei1 all group 2
pass in all
```

Rules are usually grouped by interface, but other criteria can also be used.

How Packets Are Matched against Rules on page 23 By default, a firewall inspects each packet against every rule, then takes action on the basis of the last match. In the following rule set, the firewall acts only on the basis of the final rule, allowing all packets to pass.

5.1. How Packets Are Matched against Rules

By default, a firewall inspects each packet against every rule, then takes action on the basis of the last match. In the following rule set, the firewall acts only on the basis of the final rule, allowing all packets to pass.

```
block in all
pass in all
```

In effect, the final rule in a rule set becomes the default action for the firewall.

In a large rule set, however, checking every rule and subrule can lead to long processing times and unintended results. For this reason, you may need to control rule processing.

Controlling Rule Processing

To control rule processing, include the **quick** keyword in a rule or subrule. This keyword instructs the firewall to abort processing on the first match and immediately take the action specified in the rule.

For example, in the following rule set, the firewall aborts processing with the first rule and blocks all incoming packets.

```
block in quick all
pass in all
```

Rule Processing in Grouped Rule Sets

In grouped rule sets, the **quick** keyword has a slightly different effect. If a packet fails to match a head rule, the firewall skips all subrules within that group and proceeds to the next head rule. The firewall only checks subrules when a packet meets the criteria specified by the head rule.

For the example in *4.5 Rules and Rule Groups, p.24*, if a packet arrives on **fei1** from 12.1.1.1, the firewall attempts to match it against the first head rule:

```
block in quick on fei0 all head 1
```

Because there is no match, the firewall skips the subrules in group 1 and proceeds directly to group 2.

The firewall then attempts to match the packet against the second head rule:

```
block in quick on fei1 all head 2
```

Because the packet meets the interface criterion specified in the rule, the firewall then attempts to match the packet against the subrules in group 2.

The packet fails to match the source address criteria (10.0.0.0/8 and 11.0.0.0/8), therefore it matches against the final rule (**pass in on fei1 all group 2**) and passes the firewall.

WNDRVR

Parent topic: Rules and Rule Groups on page 22

6. Rate Limiting

Rate limiting filters packets based on a specified intercept rate for a specified type of packet. You can instruct the firewall to pass or block packets of the specified type under the following conditions:

- When their rate of transmittal is greater than or less than the specified rate.
- When the number of packets transmitted or received exceeds a specified quantity during a specified period of time.

Rate limiting is available for both IP and MAC filter rules.

Rate limiting works like a token bucket filter. The burst parameter specifies the size of the bucket—that is, how many tokens the bucket can hold. The limit parameter specifies the maximum average rate at which new tokens are allowed to enter the bucket.

For example, if a rule specifies a maximum rate of 500 packets in a 120-second period, any packets exceeding that rate match the condition. If the action is to reject such packets, you can use this rule to block attempts to flood the stack with packets.

You can also rate-limit packets arriving from or being transmitted to a specified host. If you do not specify a host, the firewall limits the rate of all intercepted packets of the specified type, regardless of their source or destination address.

To limit traffic by rate, use the limit keyword with a numeric quantifier with a unit specifier to specify the unit of time. Valid units are **s** (second), **m** (minute), **h** (hour), or **d** (day).

To limit traffic by quantity—that is, to specify a maximum or minimum number of packets of a particular type—use the burst keyword. Use a numeric quantifier of at least 1.

For example, the following rule limits incoming TCP packets with SYN flags to a rate of 5 per second or bursts of ten SYN segments:

```
pass in limit 5/s burst 10 proto tcp all flags S
```

The following rule accepts 100 frames per day from MAC address 00:08:74:01:00:01:

```
pass in limit 100/d burst 1 from 00:08:74:01:00:01 to any
```

The following example logs one outgoing packet per hour:

```
pass out log limit 1/h burst 1 all
```

Note also that, as with other keywords, the exclamation point (!) can be used to invert a specified parameter. The following example blocks incoming ICMP packets unless they arrive at a rate exceeding 10 packets per second.

```
block in limit ! 10/s burst 10 proto icmp all
```

You can combine rate limiting with stateful inspection and custom processing. For further information, see *Combining Stateful Inspection with Rate Limiting or Custom Functions*

7. Logging

Logging is available for both IP packets and MAC frames. Logged information is stored in memory and can be retrieved for display on the system console.

Log Formats on page 25
Logging Traffic on page 26 To log firewall traffic, include the log keyword in the rule. For example:

7.1. Log Formats

IP Filter Logs

When logging is specified, Wind River Firewall keeps the following information for IP packets:

- Date and time the packet arrived or departed.
- Interface on which the packet arrived or departed.
- Rule group and rule index within the group.
- Action taken: passed (**p**) or blocked (**b**).
- Source IP address.
- Source port for TCP/UDP packets.
- Destination IP address.
- Destination port for TCP/UDP packets.
- Protocol.
- IP header length.
- Total length.
- ICMP type and code for ICMP packets.
- TCP flags for TCP packets.
- A notice if the packet is a fragment.

The following is an example of an IP filter log:

```
2006/11/08 16:46:49.167074 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
```

MAC Filter Logs

When logging is specified, Wind River Firewall keeps the following information for MAC frames:

- Date and time the packet arrived or departed.
- Interface on which the packet arrived or departed.
- Rule group and rule index within the group.
- Action taken: passed (**p**) or blocked (**b**).
- Source MAC address.
- Destination MAC address.
- MAC type.
- Frame length.
- First 64 bytes of frame data.

The following is an example of a MAC filter log:

```
2006/11/08 19:22:56.733333 vlan5 @0:1 p 00:a0:1e:11:11:00 -> 00:01:01:01:01:00 TYPE 8100 len 88
0005080045000054f26900003f0171d80a3201010a3202030800fa2083450003202122232425 2
62728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
```

Parent topic: Logging on page 25

7.2. Logging Traffic

To log firewall traffic, include the log keyword in the rule. For example:

```
block in log quick all
```

To reduce the number of packets stored in the log, use the **first** keyword with log. For example:

```
block in log first quick on fei0 from 10.0.0.0/8 to any
```

This rule instructs the firewall to log only the first packet arriving on **fei0** from the address space 10.0.0.0/8. Because the log can hold only a limited number of packets (100 by default), use this parameter to avoid filling up the log too fast.

Viewing the Firewall Log

To view the firewall log, type the following shell command:

```
-> cmd
[vxWorks *]# ipf -Pl
```

Clearing the Firewall Log

To clear (flush) the firewall log, type the following shell command:

```
[vxWorks *]# ipf -Fl
```

Adjusting Log Capacity

To adjust the default log capacity, edit the following macros in ipfirewall_h.h. This file is located in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/firewall-***x.x.x.x* **/h**

Maximum number of MAC filter log rules:

```
#define IPFIREWALL_MAX_MAC_LOG_ENTRIES 100
```

Maximum number of IP filter log rules:

```
#define IPFIREWALL_MAX_IP_LOG_ENTRIES 100
```

The default value for each macro is 100.

Parent topic: Logging on page 25

8. Enabling and Disabling the Firewall

To enable the firewall, type the following shell command:

```
[vxWorks *]# ipf -E
```

To disable the firewall, type the following shell command:

```
[vxWorks *]# ipf -D
```

9. Adding and Removing Firewall Rules

There are three ways to add rules to a firewall:

- By storing the rules in a file that is automatically loaded on startup (see Saving and Restoring Firewall Rules on page 29).
- By adding individual rules or an entire rule set with a shell command.
- By adding individual rules or an entire rule set with the Wind River Firewall API.

Once added, rules are stored in an internal table in system memory. By default, the firewall appends all rules to this rule set without checking for duplicates or conflicts. If you load rules on startup with a rule file, then add additional rules using a shell command or API, the firewall automatically appends them to the existing rule set.

Adding Rules on page 27 To add a rule to the firewall, type the following shell command:
Removing Rules on page 29 To remove a rule from the firewall, type:
Checking Rule Syntax on page 29 To check the rule syntax, type:

9.1. Adding Rules

To add a rule to the firewall, type the following shell command:

```
-> cmd
[vxWorks *]# ipf rule
```

Where rule is the firewall rule you wish to add. The default operation is to append the specified rule to the IP filter. For example:

```
[vxWorks *]# ipf block in quick from 192.168.1.14 to any
```

This example makes **block in quick from 192.168.1.14 to any** the last rule in an existing IP filter.

To add a rule that applies only to IPv6 packets, type:

```
[vxWorks *]# ipf -6 rule
```

Adding Rules from a File

You can also store firewall rules in a text file with a name such as **myrules.cfg**. To load all rules in this file at once, specify the file name and path of the rules file. For example:

```
[vxWorks *]# ipf -f myrules.cfg
```

If you do not specify a path, Wind River Firewall tries to open the file in the current working directory. If the file is in a different directory, you can specify the absolute path to it. For example:

```
[vxWorks *]# ipf -f /usr/local/ipfirewall/config/myrules.cfg
```

Specifying the Rule Position

By default, new rules are appended to the existing rule list. However, you can insert a rule into a list at a specified location by using the ipf shell command with the index parameter (**@**). This feature is available for both IP and MAC filters.

The index parameter must be the first parameter in the shell command. In a set of ungrouped rules, the index begins with 1 by default. Any index you specify must be 1 or greater. For example:

```
[vxWorks *]# ipf @2 block in quick from 192.168.1.15 to 10.0.0.4
```

This example inserts the rule **block in quick from 192.168.1.15 to 10.0.0.4** as the second rule in an existing rule set. If the initial rule set is as follows:

```
block in quick from 192.168.1.14 to any
block in quick from 10.0.0.4 to any
```

Then, the resulting rule set is as follows:

```
block in quick from 192.168.1.14 to any
block in quick from 192.168.1.15 to 10.0.0.4
block in quick from 10.0.0.4 to any
```

Inserting a Rule within a Group

When rule grouping is used, a new rule is inserted by default as the last rule in the specified group. If the index parameter is used in conjunction with grouping, it refers to the position of a rule within its group. Within the group, the head rule is index zero, the first rule index 1, and so on. Thus, the shell command inserts the new rule into the second position in group 9:

```
[vxWorks *]# ipf @2 block in quick on fei1 from 11.0.0.0/8 to any group 9
```

The resulting rule set is as follows:

```
block in quick on fei0 all head 8
    block in quick on fei0 from 10.0.0.0/8 to any group 8
    block in on quick fei0 from 11.0.0.0/8 to any group 8
    pass in on fei0 all group 8
block in quick on fei1 all head 9
    block in quick on fei1 from 10.0.0.0/8 to any group 9
    block in on quick fei1 from 11.0.0.0/8 to any group 9
```

```
    pass in on fei1 all group 9
pass in all
```

Parent topic: Adding and Removing Firewall Rules on page 27

9.2. Removing Rules

To remove a rule from the firewall, type:

```
[vxWorks *]# ipf -r rule
```

To remove all rules from the firewall at once, type:

```
[vxWorks *]# ipf -Fr
```

Parent topic: Adding and Removing Firewall Rules on page 27

9.3. Checking Rule Syntax

To check the rule syntax, type:

```
[vxWorks *]# ipf -n rule
```

The -n option parses the rule syntax and reports any errors without adding the rule to the firewall. When the rule syntax is correct, there is no output. When the rule syntax is incorrect, Wind River Firewall reports the error. For example:

```
[vxWorks *]# ipf -n pas in all
```

This rule returns the following error message:

```
Unknown action: pas.
```

Parent topic: Adding and Removing Firewall Rules on page 27

10. Saving and Restoring Firewall Rules

Wind River Firewall supports nonvolatile (NV) storage to the file system. To implement NV storage, save your rule set in a text file. You can use any file name or extension, but it is common to use a **.cfg** extension. The file must reside on the target and be stored on local media.

You should maintain separate rule files for IPv4, IPv6, and MAC rules. Certain rules, such as **block in all**, have the same syntax for each filter type. Segregating them in separate keyword files allows the firewall to apply the rules correctly.

This capability allows the system to save the firewall rules and restore them on system reset.

NV storage is defined by the following Workbench 4 parameters (or the related system variables):

IPF_IPV4_RULE_FILE

Specifies the name of the default IPv4 IP filter rule file. The firewall loads the rules from this file if it exists at boot time. The default is **IPCOM_FILE_ROOT"fw4.cfg"**.

IPF_IPV6_RULE_FILE

Specifies the name of the default IPv6 IP filter rule file. The firewall loads the rules from this file if it exists at boot time. The default is **IPCOM_FILE_ROOT"fw6.cfg"**.

IPF_FWMAC_RULE_FILE

Specifies the name of the default MAC filter rule file. The firewall loads the rules from this file if it exists at boot time. The default is **IPCOM_FILE_ROOT"fwmac.cfg"**.

On restart, the system loads the rules in the files defined by these parameters.

11. Viewing and Clearing Firewall Information

Wind River Firewall maintains information on firewall rules and operations in the form of logs, tables, and statistics. You can retrieve this information by typing the appropriate shell command.

For information on accessing and clearing firewall logs, see *Logging Traffic on page 26*

Viewing and Clearing Firewall Statistics on page 30 Wind River Firewall keeps the following statistics:
Viewing and Clearing Firewall Tables on page 31 Wind River Firewall keeps the following tables:

11.1. Viewing and Clearing Firewall Statistics

Wind River Firewall keeps the following statistics:

- Input packets (blocked, passed, and nonmatching).
- Output packets (blocked, passed, and nonmatching).
- Invalid packets.
- Logged input packets (blocked and passed).
- Logged output packets (blocked and passed).
- Log failures.
- States added.
- States expired.
- State hits.
- State failures.
- Input MAC frames (blocked, passed, and nonmatching).
- Output MAC frames (blocked, passed, and nonmatching).
- Invalid MAC frames.
- Logged input MAC frames (blocked and passed).
- Logged output MAC frames (blocked and passed).
- MAC log failures.

To display these statistics, type the following shell command:

```
-> cmd
[vxWorks *]# ipf -S
```

The following is an example of statistics kept by the firewall:

```
FIREWALL STATISTICS:
input packets: blocked 0 passed 0 nomatch 0
output packets: blocked 0 passed 0 nomatch 0
invalid packets: 0
logged input packets: blocked 0 passed 0
logged output packets: blocked 0 passed 0
log failures: 0
states added: 0
states expired: 0
state hits: 0
state failures: 0
input mac frames: blocked 0 passed 0 nomatch 0
output mac frames: blocked 0 passed 0 nomatch 0
invalid mac frames: 0
logged input mac frames: blocked 0 passed 0
logged output mac frames: blocked 0 passed 0
mac log failures: 0
```

To clear the statistics, type the following:

```
[vxWorks *]# ipf -Z
```

Parent topic: Viewing and Clearing Firewall Information on page 30

11.2. Viewing and Clearing Firewall Tables

Wind River Firewall keeps the following tables:

- Rules.
- State entries.
- Log.
- User (custom functions).

```
[vxWorks *]# ipf -Pa
```

To clear all tables, type the following:

```
[vxWorks *]# ipf -Fa
```

Rule Table

To view the rule table, type the following:

```
[vxWorks *]# ipf -Pr
```

The following is an example of a firewall rule table:

```
IP FILTER RULE TABLE:
AF_INET: @1 pass out quick on vlan5 all group 0:1
AF_INET: @2 block out on vlan5 all group 0:2
```

To view all rules for a specific group, type the following:

```
[vxWorks *]# ipf -PgN
```

Where *N* is the number of the group whose rules you want to view.

To clear the rule table, type the following:

```
[vxWorks *]# ipf -Fr
```

To clear all rules for a specific group, type the following:

```
[vxWorks *]# ipf -FgN
```

Where *N* is the number of the group whose rules you want to clear.

State Table

To view the state table, type the following:

```
[vxWorks *]# ipf -Ps
```

The following is an example of a state table:

```
IP FILTER STATE TABLE:
10.50.1.1:40000 -> 10.50.2.3:30000 proto udp expire in 56 s
10.50.1.1:40000 -> 10.50.2.3:30000 proto tcp expire in 58 s
10.50.1.1:21217 -> 10.50.2.3:21217 proto icmp expire in 3 s
```

To clear the state table, type the following:

```
[vxWorks *]# ipf -Fs
```

Log Table

To view the log table, type the following:

```
[vxWorks *]# ipf -Pl
```

The following is an example of a log table:

```
IP FILTER LOG:
2006/11/08 16:46:49.167074 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
2006/11/08 16:46:50.168399 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
2006/11/08 16:46:51.172488 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
```

```
2006/11/08 16:46:52.176583 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
2006/11/08 16:46:53.180652 vlan5 @0:1 p 10.50.1.1 -> 10.50.2.3 PR icmp len 20 84 icmp 8/0
```

To clear the log table, type

```
[vxWorks *]# ipf -Fl
```

Custom Functions Table

For information on viewing custom functions, see *Viewing Custom Functions on page 49*

Group Rule Table

To view the rule table for a particular group, type:

```
[vxWorks *]# ipf -PgN
```

Where *N* is the number of the group whose rule table you want to view.

For example:

```
[vxWorks *]# ipf -Pg2
```

The following is an example of the table generated by the previous command:

```
IP FILTER RULE TABLE:
AF_INET: @1 pass in proto icmp all head 1:0
AF_INET: @2 pass in proto igmp all group 1:1
AF_INET: @3 pass in proto 3 all group 1:2
```

To clear the rule table for a particular group, type:

```
[vxWorks *]# ipf -FgN
```

Where *N* is the number of the group whose rule table you want to clear.

For example:

```
[vxWorks *]# ipf -Fg2
```

Parent topic: Viewing and Clearing Firewall Information on page 30

5. CREATING AN IP FILTER

1. Introduction

The IP filter operates in the network layer of the TCP/IP stack. It filters IPv4 or IPv6 packets based on the rules you specify. You can write rules to filter incoming or outgoing packets, using any combination of source addresses, destination addresses, or fields in a packet header.

Inbound traffic as specified with the **in** keyword is all traffic that enters the firewall on any interface. Outbound traffic as specified with the **out** keyword is all traffic that leaves the firewall on any interface. Hence, all packets are filtered as they enter the firewall and as they exit the firewall.

You should have rules for every direction and every interface.

By default, the firewall inspects each packet against every rule for a given direction, then takes action on the basis of the last match. To control rule processing and reduce processing time, include the **quick** keyword in a rule or subrule. This keyword instructs the firewall to abort processing on the first match and immediately take the action specified in the rule. Hence, when using the **quick** keyword, the order in which the *pass* and *block* rules are listed is important.

The *keep state* keywords allow the firewall to focus on blocking or passing new connections instead of filtering on every packet. Use the **keep state**, **keep state limited**, **keep ifstate**, or **keep ipsecstate** keywords to filter on connections.

This chapter describes filtering methods specific to the IP filter (see *Methods for Filtering on page 34*). You should also review the following chapters:

- Fundamentals on page 19 (for information on filtering methods available for both IP and MAC filters).
- Filtering HTTP Content on page 50 (for information on HTTP filters).

2. Methods for Filtering

You can filter IP packets by a variety of means, including the following:

- Address.
- Type of service or traffic class.
- Time to live.
- Protocol.
- ICMP type and code.
- Port specification for UDP and TCP protocols.
- TCP flags.
- IP options and fragments.

Filtering by Address on page 35 You can instruct the firewall to pass or block individual addresses or a range of addresses.
Filtering by Type of Service or Traffic Class on page 35 You can filter packets based on a value in the type of service (**tos**) field in the packet header. Examples include:
Filtering by Time to Live on page 36 You can filter packets based on a value in the time to live (**ttl**) field in the packet header. For example:
Filtering by Protocol on page 36 You can filter packets based on their Internet protocol, using either the protocol name or a numeric value. A special case protocol is tcp/udp, which means either TCP or UDP. Examples include:

WNDRVR

2.1. Filtering by Address

You can instruct the firewall to pass or block individual addresses or a range of addresses.

IP Filter Address Scope

The address scope parameter specifies an individual address or range of addresses for either the source or destination of the data packet. The keyword **all** specifies all traffic, regardless of source or destination address. The keyword **me** specifies any address configured on the system. The keyword **from** precedes a source address. The keyword **to** precedes a destination address. When used with from, the keyword **any** specifies any source address. When used with **to, any** specifies any destination address. Examples include:

```
block in all
pass out from me to any
```

Specific addresses are shown as follows:

```
block in from 192.168.1.14 to any
```

To specify a range of addresses, use the / operator. For example:

```
block in from 10.0.0.0/8 to any
```

This example blocks packets from address space 10.0.0.0 to 10.255.255.255 from passing through the firewall and reaching a destination address.

Preceding an address or range of addresses with an exclamation point (!) inverts the specification. For example:

```
pass in from ! 192.168.1.14 to me
```

This example permits packets from any address except 192.168.1.14 to pass the firewall and reach the system.

IPv6 Addresses

IPv6 addresses are specified as follows:

```
pass out from 3ffe::b80:2:6cbf to any
pass out from 3ffe:b80:a19::/48 to any
```

Parent topic: Methods for Filtering on page 34

2.2. Filtering by Type of Service or Traffic Class

You can filter packets based on a value in the type of service (**tos**) field in the packet header. Examples include:

```
block in tos 0x0c all
block in tos ab all
```

A **tos** mask can be used to specify that certain bits must match while others can differ. The **tos** mask is combined with the tos field in the incoming packet header (using an AND operator), and the resulting value is compared with the **tos** value in the rule. Examples include:

WNDRVR

```
block in tos 0x80/0xe0 all

block in tos F0/FC all
```

The **tos** value and **tos** mask must be specified in hexadecimal. A leading 0x or 0X is optional. A slash (/) separates the **tos** value specification from the **tos** mask specification.

Parent topic: Methods for Filtering on page 34

2.3. Filtering by Time to Live

You can filter packets based on a value in the time to live (**ttl**) field in the packet header. For example:

```
block in ttl 0 all
```

Parent topic: Methods for Filtering on page 34

2.4. Filtering by Protocol

You can filter packets based on their Internet protocol, using either the protocol name or a numeric value. A special case protocol is tcp/udp, which means either TCP or UDP. Examples include:

```
block in proto icmp all
pass out proto udp all
pass in proto 89 all
```

Filtering by ICMP Type and Code

If the protocol is ICMP, you can specify the **icmp-type** and **code**, using a numeric value. Examples include:

```
block in proto icmp all icmp-type 8
block out proto icmp all icmp-type 0 code 0
```

Filtering by Port for UDP and TCP Protocols

For UDP and TCP protocols, you can specify the source port, destination port, or both. The port can be an individual port or an interval. For example:

```
block in quick proto tcp from any to any port = 80
```

If the **port** keyword precedes the **to** keyword, the specification refers to a source port. If the **port** keyword follows the **to** keyword, the specification refers to a destination port. Consider the following three examples.

The first rule blocks all TCP traffic bound for destination port 80:

```
block in quick proto tcp from any to any port = 80
```

The second rule passes outbound UDP traffic from any source port below 1024:

```
pass out log proto udp from any port < 1024 to any
```

The third rule passes outbound TCP traffic from any source port below 1024 to destination port 80:

```
pass out log proto tcp from any port < 1024 to any port = 80
```

A variety of mathematical operators are available for use with the **port** keyword. For further information, see *A.3.34 port on page 68*.

Filtering by TCP Flags

If the protocol is TCP, you can specify certain TCP flags in the rule, which are matched against the TCP flags in the packet header. Valid settings include:

- U (URG)
- A (ACK)
- P (PSH)
- R (RST)
- S (SYN)
- F (FIN)
- 0 (no flags must be active)

If you specify a particular flag, that flag must be set in the packet header for the rule to match. Flags that are not explicitly specified must not be set. For example:

```
block in quick proto tcp all flags S
```

This example requires the **S** flag to be set in the packet header while all other flags are cleared. Conversely, the following line requires that no flags are set:

```
block in quick proto tcp all flags 0
```

You can also specify a flag mask, which requires a match on certain flags while allowing flexibility on others. A slash (**/**) separates the flag type specification from the flag mask specification. All flags specified before the slash must be set in the TCP header for the rule to match. All flags not specified before the slash must not be set in the TCP header for the rule to match.

Flags specified after the slash must strictly conform to the specification set before the slash. Flags not specified after the slash are allowed to vary from the specification set before the slash.

For example:

```
block in quick proto tcp all flags S/SA
```

This example indicates that the header must conform to the specification for the **S** and **A** flags--set for **S** and cleared for **A**. Other flags can be either set or cleared, and the rule is still a match.

The practical effect is that the default mask UAPRSF (meaning that all flag bits must be set) is in effect even if you do not explicitly specify it in the rule. Thus, the two rules below are equal:

```
    block in proto tcp all flags S

    block in proto tcp all flags S/UAPRSF
```

The default mask is **UAPRSF**. That means that if no flag mask is specified, all flags must match their type specification.

WNDRVR

Filtering by IP Options and Fragments

You can use the **with** keyword to filter packets including IP options and fragments. The **ipopts** keyword is only relevant for IPv4 rules, while the **frag** keyword is relevant for both IPv4 and IPv6 rules. To exclude packets with fragments or IP options from a specification, use **with no**. Examples include:

```
block in quick all with frag
pass in log quick all with ipopts
pass in all with no frag
```

Filtering by Packet Flags

You can use the **pktflags** keyword to filter packets based on the presence of certain packet flags. This keyword supports the filtering of IPsec, NAT, and tunneled packets. The **pktflags** keyword can only be used in conjunction with input rules.

Valid settings include:

- **S** (packet was processed by IPsec)
  N (packet was processed by NAT)
  T (packet was tunneled)
  0 (none of the above)

The sample rule set below blocks all incoming packets that were not processed by IPsec transport or IPsec tunnel mode:

```
pass in quick pktflags S all
pass in quick pktflags ST all
block in all
```

You can also specify a flag mask, which requires a match on certain flags while allowing flexibility on others. A slash (**/**) separates the flag type specification from the flag mask specification. All flags specified before the slash must be set in the packet header for the rule to match. All flags not specified before the slash must not be set in the packet header for the rule to match.

Flags specified after the slash must strictly conform to the specification set before the slash. Flags not specified after the slash are allowed to vary from the specification set before the slash.

The sample rule set below blocks all incoming packets that were not processed by IPsec in transport mode. IPsec tunnel mode packets are blocked because of the presence of **T** in the mask.

```
pass in quick pktflags S/ST all
block in all
```

Parent topic: Methods for Filtering on page 34

3. Stateful Inspection

Stateful inspection analyzes the transport layer headers in data packets to track the state of network connections. Using this header information, stateful inspection identifies whether each packet is a new connection request or a packet belonging to a previously established connection. You can write filtering rules to pass or block packets based on the state information. Stateful inspection keeps the state for TCP packets, UDP packets, ICMP echo packets, and ICMPv6 echo packets.

Wind River Firewall stateful inspection requires a rule with a **keep state** keyword to create a state tracking entry for outgoing packets. When an outgoing packet matches the rule, the firewall temporarily opens a port for packets arriving in response to such a request.

The firewall then matches incoming packets against active state entries before checking other rules. If there is a matching state entry, the firewall bypasses other rules and accepts the packet. If the state entry has timed out, the packet is tested against the remaining rules in the rule set.

The state tracking entry contains the following information:

- Source and destination IP addresses.
- Source and destination ports for UDP and TCP.
- ICMP ID and sequence number for ICMP or ICMPv6 echo.
- Interface (optional).
- Encryption by IPsec (optional).

Use the **keep state** keyword to create a state tracking entry. If you want to include the interface in the state tracking entry, use the **keep ifstate** keyword instead of the **keep state** keyword and specify the source port identifier of the interface.

If you want the firewall to track the state of an existing connection based on encryption by IPsec, use the keep **ipsecstate** keyword. For outgoing packets, if encryption is in use, the VPN gateway address is stored in the firewall state entry. For incoming packets, the packet must be issued from the same VPN gateway before it is forwarded to the destination.

Using the keep state Keyword

The following example allows DNS responses to pass the firewall in response to an outgoing connection request transmitted on port 53:

```
pass out proto udp from any to any port = 53 keep state
```

Using the keep ifstate Keyword

The following example allows DNS responses to pass the firewall in response to an outgoing connection request transmitted on port 53 from interface **fei0**.

```
pass out proto udp from fei0 to any port = 53 keep ifstate
```

Using the keep ipsecstate Keyword

If you want to track the state of a connection based on the use of encryption by IPsec, the rule is similar to the following:

```
pass out proto udp from any to any keep ipsecstate
```

Configuring Stateful Inspection on page 39 Use the firewall configuration parameter **IPF_MAX_STATEFUL_MAPPINGS** to specify the maximum number of stateful mappings the firewall can handle. The default is 1,000.
Combining Stateful Inspection with Rate Limiting or Custom Functions on page 40 You can combine stateful inspection with rate limiting, custom functions, or both.

3.1. Configuring Stateful Inspection

Use the firewall configuration parameter **IPF_MAX_STATEFUL_MAPPINGS** to specify the maximum number of stateful mappings the firewall can handle. The default is 1,000.

Use one of the following firewall parameters to specify stateful mapping timeout values for individual protocols:

- **IPF_ICMP_TIMEOUT**
- **IPF_UDP_TIMEOUT**
- **IPF_TCP_TIMEOUT**
- **IPF_OTHER_TIMEOUT**

For further information on these parameters, see *Wind River Firewall Configuration Parameters on page 7*.

Parent topic: Stateful Inspection on page 38

3.2. Combining Stateful Inspection with Rate Limiting or Custom Functions

You can combine stateful inspection with rate limiting, custom functions, or both.

You can apply rate limiting rules to a packet that has already matched a state entry in the firewall. For example, the following rule limits ICMP reply packets that match an existing state entry. Specifically, the rule limits ICMP reply packets on **fei0** to a rate of ten packets per second in bursts of ten packets.

```
pass out limit 10/s burst 10 on fei0 proto icmp all keep state
```

You can also apply your own custom functions to a packet that matches a state entry. For example, the following rule applies custom processing defined in a function named **userdef_example**:

```
pass out on fei0 proto icmp all keep state userdef userdef_example
```

Finally, you can apply both rate limiting rules and user-defined rules to the same packet. For example:

```
pass out limit 10/s burst 10 on fei0 proto icmp all keep state userdef userdef_example
```

For further information on rate limiting, see *Rate Limiting on page 24*. For further information on custom functions, see *Defining Custom Functions on page 48*.

Parent topic: Stateful Inspection on page 38

4. Responding to Blocked Packets

Normally, packets are silently dropped at the firewall when blocked. Because this behavior is sometimes undesirable, there are three keywords available for instructing the firewall to send a response back to the peer. These keywords are only available for IP filter rules and not MAC filter rules.

Sending a Reset Segment (TCP Only)

The **return-rst** keyword can be used to send a reset segment back to the peer if a packet is blocked. This option is only available for the TCP protocol and is useful if certain services are blocked by the firewall. If this option is used, the peer will receive a connection refused error instead of a connection timeout. For example:

```
block in return-rst proto tcp from any to any port = 80
```

Sending a Destination Unreachable Message (ICMP Only)

The **return-icmp** keyword can be used to send an ICMP destination unreachable message back to the peer. Any ICMP destination unreachable code can be used, but the default is **unreachable-network**. The **unreachable-port** code must be used if the firewall blocks a packet destined for a UDP port. Some examples are as follows:

- Block incoming packets addressed to the 11.0.0.0/8 address space and send an ICMP network unreachable code to the peer:

```
block in return-icmp from any to 11.0.0.0/8
```

- Block incoming UDP packets on port 53 and send an ICMP port unreachable code to the peer:

```
block in return-icmp(3) proto udp from any to any port = 53
```

The **return-icmp-as-dest** keyword is used like **return-icmp**, except that the ICMP message is sent with a source address copied from the destination address of the blocked packet. This has the advantage that the peer sees only the remote host address and not the firewall. For example:

```
block in return-icmp-as-dest(3) proto udp from any to any port = 53
```

All three keywords are also available for IPv6 rules. An ICMPv6 message is sent for the **return-icmp** and **return-icmp-as-dest** keywords. The default unreachable code is **no-route** for IPv6 rules.

6. CREATING A MAC FILTER

1. Introduction

The MAC filter operates in the data link layer of the TCP/IP stack. It filters packets based on the rules you specify. You can write rules to filter incoming or outgoing packets, using any combination of source MAC address, destination MAC address, interface, frame type, and packet length.

Inbound traffic as specified with the **in** keyword is all traffic that enters the firewall on any interface. Outbound traffic as specified with the **out** keyword is all traffic that leaves the firewall on any interface. Hence all packets are filtered as they enter the firewall and as they exit the firewall.

You should have rules for every direction and every interface.

By default, the firewall inspects each packet against every rule, then takes action on the basis of the last match. To control rule processing and reduce processing time, include the **quick** keyword in a rule or subrule. This keyword instructs the firewall to abort processing on the first match and immediately take the action specified in the rule. Hence, when using the **quick** keyword, the order in which the pass and block rules are listed becomes important.

This chapter describes filtering methods specific to the MAC filter (see *Methods for Filtering on page 42*). For filtering methods available for both MAC and IP filters, see *Fundamentals on page 19*.

2. Methods for Filtering

You can filter MAC packets using the following methods:

- Address.
- Interface.
- Frame type.

Filtering by Address on page 42 The address scope parameter for MAC filters is similar to that used for IP filters. This parameter can specify an individual address or range of addresses for either the source or destination of the data frame.
Filtering by Interface on page 43
Filtering by Frame Type on page 43 The **mac-type** keyword can be used to filter frame types, based on the MAC type specified in the frame header. For example:

2.1. Filtering by Address

The address scope parameter for MAC filters is similar to that used for IP filters. This parameter can specify an individual address or range of addresses for either the source or destination of the data frame.

The keyword **all** specifies all traffic, regardless of source or destination address. The keyword **me** specifies the MAC address assigned to the interface the frame is sent or received on. The keyword **from** precedes a source address. The keyword to precedes a destination address. When used with **from**, the keyword **any** specifies any source address. When used with **to, any** specifies any destination address. Examples include:

```
pass out from me to any
block in from any to me
```

Specific addresses are shown as follows:

WNDRVR

```
block in from 00:08:74:00:00:01 to any
```

A range of addresses is shown as follows:

```
block in from 00:08:74:01:00:00/FF:FF:FF:FF:00:00 to any
block in from any to 00:AO:88:11:00:00/FF:FF:FF:FF:00:00
```

Preceding an address or range of addresses with an exclamation point (!) inverts the specification. For example:

```
block in from any to ! me
```

This example blocks packets from any source address to all destination addresses except the one assigned to the interface the packet is received on.

Parent topic: Methods for Filtering on page 42

2.2. Filtering by Interface

Filtering by interface is discussed in *Interface on page 21*.

Parent topic: Methods for Filtering on page 42

2.3. Filtering by Frame Type

The **mac-type** keyword can be used to filter frame types, based on the MAC type specified in the frame header. For example:

```
block in on fei0 all mac-type 0x86DD
block in from 00:08:74:01:00:01 to me mac-type 0x0806
```

Parent topic: Methods for Filtering on page 42

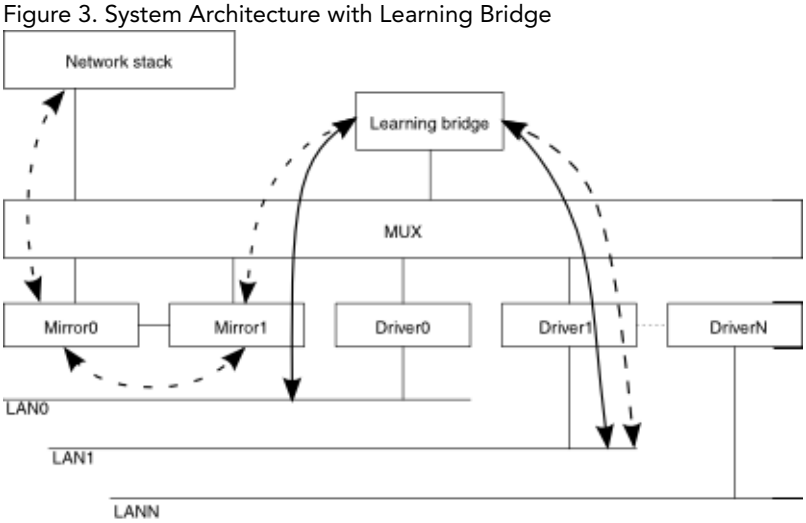
7. FILTERING TRAFFIC IN THE WIND RIVER LEARNING BRIDGE

1. Introduction

This chapter describes how to configure the firewall so that it filters MAC frames or IP packets at the learning bridge.

If you plan to implement a firewall in the learning bridge, you should visualize the architecture of the learning bridge and network stack and the paths of data packets transiting the system. The *System Architecture with Learning Bridge on page 44* figure illustrates this architecture and traffic flow.

Figure 3. System Architecture with Learning Bridge



The learning bridge occupies a position above the MUX. Packets traveling between **LAN 0** and **LAN 1** (represented by a solid line) transit the MUX and the learning bridge without entering the network stack. With firewall support enabled in the learning bridge, you can write firewall rules to filter these packets in the learning bridge. This process is called port-to-port filtering.

Packets traveling between **LAN 1** and the network stack (represented by a dotted line) transit the MUX, the learning bridge, and the virtual drivers **Mirror 0** and **Mirror 1**. As in the previous case, you can write firewall rules to filter these packets at this location, provided you have firewall support enabled in the learning bridge. This process is called port-to-stack filtering, or stack-to-port, depending on the direction of packet flow. In this case, packets bypass the firewall in the network stack to avoid redundant processing.

Enabling Firewall Support in the Learning Bridge on page 44 To filter network traffic in the learning bridge, you must first build the required libraries.
Enabling Firewall Support in a Bridge on page 45

1.1. Enabling Firewall Support in the Learning Bridge

To filter network traffic in the learning bridge, you must first build the required libraries.

To do so, enable **IPNET_BRIDGE** and **IPFIREWALL_SUPPORT** in your VxWorks source build project, as described in *Adding Firewall Support in the Learning Bridge on page 6*

Parent topic: Introduction on page 44

1.2. Enabling Firewall Support in a Bridge

To provide firewall support for a bridge and enable the phys option for the ipf block and ipf pass commands, see Source Build Configuration Options on page 6.

The phys option specifies a physical interface to be used in a filter rule when using a bridge.

Parent topic: Introduction on page 44

2. Configuring the Firewall at the Learning Bridge

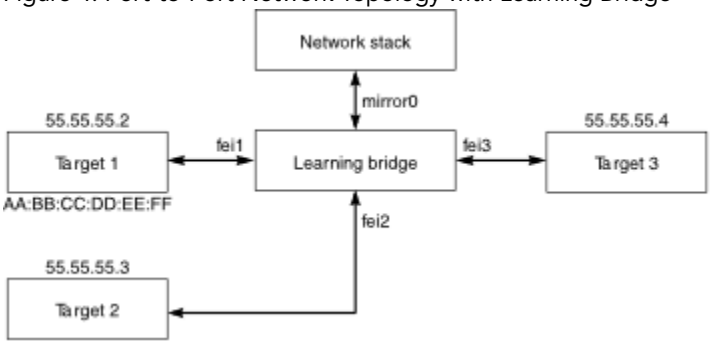
You can configure the firewall to process frames in the learning bridge, using either a MAC filter (at layer 2) or an IP filter (at layer 3, 4, or 5; depending on the contents of the packet's IP header). The firewall can filter traffic that only transits the learning bridge (port-to-port filtering), or it can filter traffic that transits the learning bridge and the network stack (port-to-stack or stack-to-port filtering).

Port-to-Port Filtering on page 45
Port-to-Stack Filtering on page 46

2.1. Port-to-Port Filtering

To visualize port-to-port filtering, consider the network topology illustrated in the Port-to-Port Network Topology with Learning Bridge on page 45 figure.

Figure 4. Port-to-Port Network Topology with Learning Bridge



Filtering IP Packets

To filter IP packets, use rules such as the following:

```
block in all
pass in from 55.55.55.2 to any
```

These rules allow only IP packets from **Target 1** to cross the learning bridge. Packets traveling from **Target 1** to **Target 2** or **Target 3** cross only the learning bridge without touching the network stack. Packets from **Target 1** can also reach the network stack on **mirror0**. Although the bridge does not typically inspect anything beyond the MAC layer, the firewall still detects the IP payload and filters it accordingly.

If a packet does not match a filter rule, it continues to its destination without transiting the network stack.

Filtering MAC Frames

To filter MAC frames, use rules such as the following:

```
-m block in on fei1 all

-m pass in on fei1 from AA:BB:CC:DD:EE:FF to any
```

Again, these rules allow only IP packets from **Target 1** to cross the learning bridge. Frames travel only across the learning bridge without touching the network stack.

As in the previous example, if a frame does not match a filter rule, it continues to its destination without transiting the network stack.

Filtering LLC/SNAP Traffic

You can also filter logical link control/subnetwork access protocol (LLC/SNAP) frames in the network stack.

The firewall detects an LLC/SNAP header by examining the **EtherType** field in a MAC frame. If the **EtherType** field is less than or equal to the maximum 802.3 frame length, the firewall assumes that an LLC header is to follow. If the **EtherType** field is greater than the maximum 802.3 frame length, the firewall interprets the field as a specification of the protocol type and assumes that the payload is to follow.

You can filter frames with LLC headers by their SSAP and DSAP values. The firewall detects a SNAP header if the DSAP and SSAP values in the LLC header are both **0xaa**. You can further filter LLC frames with SNAP headers by organization code and type.

To filter frames by their DSAP or SSAP values or by SNAP type, specify the appropriate keywords and values in your firewall rules. For example, the following rule blocks all inbound MAC frames with LLC headers:

```
block in all 802_2
```

The following rule passes all inbound MAC frames with an SSAP value of **0xaa**:

```
pass in all 802_2-ssap 0xaa
```

The following rule blocks all inbound MAC frames with a SNAP-type value of **0x0806**:

```
block in all 802_2-snap-type 0x0806
```

The following rule passes in all inbound MAC frames that are addressed to **01:80:63:07:00:00**, provided they have a DSAP value of **0xaa**, an SSAP value of **0xaa**, a SNAP-vendor value of **0x008063**, and a SNAP-type value of **0x0800**:

```
pass in quick from any to 01:80:63:07:00:00 802_2-dsap 0xaa 802_2-ssap 0xaa 802_2-snap-vendor 0x00
8063 802_2-snap-type 0x0800
```

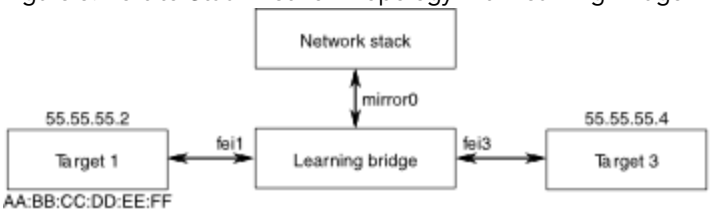
Because this rule also contains the **quick** keyword, processing halts at this point, and the firewall acts directly on the rule.

Parent topic: Configuring the Firewall at the Learning Bridge on page 45

2.2. Port-to-Stack Filtering

To visualize port-to-stack filtering, consider the network topology illustrated in Port-to-Stack Network Topology with Learning Bridge on page 47.

Figure 5. Port-to-Stack Network Topology with Learning Bridge



You can filter frames or packets traveling inbound from the learning bridge towards the network stack on this host based upon the physical bridge port. To do so, use the **phys** keyword, as in the following rule:

```
block in on mirror0 phys fei1 all
```

This rule blocks all incoming IP traffic destined to the network stack from interface **fei1**, but it allows all incoming IP traffic on **fei2** to reach the network stack.

Parent topic: Configuring the Firewall at the Learning Bridge on page 45

8. DEFINING CUSTOM FUNCTIONS

1. Defining Custom Functions

This chapter describes the process of creating custom functions, which can be used to extend the capabilities of the firewall. Some useful applications of this capability include:

- Scanning the contents of application data.
- Changing a field in the packet headers or data.
- Creating a rule to match parameters not covered by the current syntax.

Elements of a Custom Function

A custom function consists of three function hooks: a check function, a match function, and a destroy function. They have the following prototypes:

```
typedef int (* Ipfirewall_userdef_check) (void *cookie, void *info, unsigned
int infolen);
```

```
typedef int (* Ipfirewall_userdef_match) (Ipcom_pkt *pkt, void *cookie, void
*info);
```

```
typedef void (* Ipfirewall_userdef_destroy) (void *cookie, void *info,
unsigned int infolen);
```

The check function is optional and is called at the time the rule is added to the firewall. The purpose of the check function is to verify the rule parameters and possibly convert the rule parameter string to a custom type to speed up the match function's processing time.

The match function is mandatory and is called at rule matching. When all rule parameters match the packet and a custom function is specified with the rule, the match function is called.

The destroy function is optional and is called at the time the rule is removed from the firewall. The purpose of the destroy function is to free resources that were dynamically allocated by the check function.

The pkt parameter is a pointer to the complete packet. The custom function can access any field in the packet headers or packet payload.

The cookie parameter is assigned at registration of the custom function and kept through calls to the check, match, and destroy functions.

The info parameter is set to an ASCII string with the custom rule parameters. If no parameters are assigned to the rule, the info parameter will be an empty string.

The infolen parameter is set to the buffer length of the info parameter. It shows how much space is available for altering the info buffer.

The check and match functions can alter the contents of the **info** buffer. The only requirement is that the buffer size is not exceeded. The check function typically converts the ASCII string received in the call to a custom structure, which is easier for the match function to parse.

The check function must return zero (**0**) or one (**1**). Zero means that the check failed and that the rule shall not be added.

One means that the rule was successfully verified and the rule will be added. The match function must return zero (**0**) or one (**1**). Zero means that packet did not match the rule, while one means that it matched.

The functions ipfirewall_register_userdef( ) and ipfirewall_unregister_userdef( ) are available for registration and unregistration of the check and match functions. A custom function is registered with an ASCII string identifier, a mandatory pointer to the match function, an optional pointer to the check function, an optional pointer to the destroy function, and an optional cookie. The identifier is used in the rule syntax to specify the custom functions. See the following examples of rule syntax:

- The following rule specifies that the check and match functions, which have been registered with the string **scan_appdata**, are called for incoming packets on interface **fei1**:

  ```
  pass in on fei1 all userdef scan_appdata
  ```

  Because no parameters are given to **scan_appdata** in the rule syntax, the call to the check function is made with an empty string in the info parameter.
- The following rule specifies that the check and match functions, which have been registered with the string set_mprio, will be called for outgoing packets on interface **fei0**:

  ```
  pass out on fei0 all userdef set_mprio 1
  ```

  The call to the check function will be made with the **1** in the **info** parameter.

Custom functions are available for both IP filter and MAC filter rules.

You can use custom functions in combination with stateful inspection and rate limiting. For further information, see Combining Stateful Inspection with Rate Limiting or Custom Functions on page 40.

Viewing Custom Functions

To view custom—or user-defined—functions, type the following shell command:

```
-> cmd
[vxWorks *]# ipf -Pu
```

This command outputs the identifier of the match, check, and destroy functions defined in custom functions, as follows:

```
IPFIREWALL USER TABLE:
 id=userdef_example
 id=http_filter
```


WNDRVR

9. FILTERING HTTP CONTENT

1. Introduction

You can establish content filtering for HTTP traffic through Wind River Firewall, specifying filters for URLs, proxy traffic, cookies, Java applets, and ActiveX controls. Establishing such filters is typically a three-step process:

1. Define an HTTP filter and add it to the firewall.
2. Enable a particular content filter.
3. Add a rule that refers to the HTTP filter.

The first two steps require the use of the Wind River Firewall API. The third step can be performed using any of the methods available for adding firewall rules.

2. Enabling HTTP Content Filtering

Each HTTP filter is identified by an ASCII string that defines the name of the filter. Use ipfirewall_http_add_filter( ) to add HTTP filters to the firewall. This function takes the name of the filter as the only input parameter. Once you have added at least one HTTP filter, you can specify a rule that refers to it. For example, if you add an HTTP filter with the name **badurls**, you can then add a firewall rule that lets all HTTP traffic go through the filter as follows:

```
block out proto tcp from any to any port = 80 all userdef http_filter badurls
```

Before the HTTP filter actually enters operation, you must enable at least one of the available content filters. The HTTP filter checks these content filters in the following order:

1. URL filter.
2. Proxy filter.
3. Java filter.
4. ActiveX filter.
5. Cookie filter.

If a packet matches any of the filters, the entire rule is considered a match, and the HTTP filter mechanism stops checking the other filters. However, the cookie filter is always checked even if a packet matches one of the previous content filters. This check occurs because the cookie filter actually changes the content of the packet.

3. Filtering Content by URL

Understanding the URL Filter Mechanism on page 51 Wind River Firewall lets you establish and maintain a database of specific URLs or keywords that appear in URLs. The URL filter matches the URL in the packet with the database of specific URLs and keywords.You can also list acceptable URLs that the firewall allows to pass.
Implementing a URL Filter on page 51 Use ipfirewall_http_insert_url_filter( ) to add URLs and keywords to the HTTP filter. This function takes three arguments:

3.1. Understanding the URL Filter Mechanism

Wind River Firewall lets you establish and maintain a database of specific URLs or keywords that appear in URLs. The URL filter matches the URL in the packet with the database of specific URLs and keywords.You can also list acceptable URLs that the firewall allows to pass.

You first create the database, then add URLs and keywords you want to filter. After creating the database, you enable URL filtering by registering it and specifying the filter's action.

The URL filter has two filtering features: path match and keyword match.

Path Matching

Path matching compares the provided path (host name and file path) to the initial portion of the absolute path for an HTTP packet. For example, if you add the path *www.somewebsite.com* to the URL list, the firewall treats all of the following URLs as matches for the string:

- *www.somewebsite.com*
- *www.somewebsite.com/bad*
- *www.somewebsite.com/bad/a.html*

For path matching, supply at least the host name part.

Keyword Matching

Keyword matching means matching any word in the URL. For example, if you add the keyword **bad** to the URL list, the firewall considers the last two of the three URLs in the path matching example above as a match.

Parent topic: Filtering Content by URL on page 50

3.2. Implementing a URL Filter

Use ipfirewall_http_insert_url_filter( ) to add URLs and keywords to the HTTP filter. This function takes three arguments:

- The name of the HTTP filter on which you want to enable a URL filter.

  The URL or keyword to be added to the list.

  A boolean used to specify whether a URL or a keyword is added. **FALSE** specifies a URL filter, **TRUE** a keyword filter.

Parent topic: Filtering Content by URL on page 50

4. Filtering Proxy Traffic

Proxy web servers sit between your web browser and the actual web server. The use of proxy servers may allow users to circumvent the firewall's content filtering. A proxy filter matches HTTP packets sent to proxy web servers.

The proxy filter matches HTTP packets sent to an HTTP proxy server on the same destination port to which you attach the proxy content filter.

According to the HTTP version 1.1 specification (see RFC 2616, Hypertext Transfer Protocol -- HTTP/1.1, section 5.1.2), HTTP clients only generate an absolute uniform resource identifier (URI) in requests to proxies.

The URI in the HTTP request line has the following format:

```
http://www.nnn.nnn/aaa
```

A direct HTTP request line generally has only the path information (*/aaa*), without including host information. The Wind River Firewall proxy filter uses this characteristic to differentiate direct HTTP traffic from proxy traffic.

Use ipfirewall_http_insert_url_filter( ) to enable a proxy filter. This function has one input argument—the name of the HTTP filter on which you want to enable the proxy content filter.

5. Filtering Java Applets

A web client triggers a Java applet by reacting to either the **HTML APPLET** tag (for HTML version 1.0) or the **OBJECT** tag (for HTML version 1.1) in an incoming HTML page. Both tags specify the name of the Java applet class object or **.jar** file.

To match Java applets and ActiveX controls, Wind River Firewall matches the **GET** requests for files with **.class** and **.jar** extensions.

Use ipfirewall_http_insert_java_filter( ) to enable a Java filter. This function has one input argument—the name of the HTTP filter on which you want to enable the Java content filter.

6. Filtering ActiveX Controls

A web client runs an ActiveX control by reacting to the embedded file specified by an **OBJECT** tag in an incoming HTML page.

To match ActiveX controls, Wind River Firewall matches the **GET** requests for files with **.cab** and **.ocx** extensions.

Use ipfirewall_http_insert_activex_filter( ) to enable an ActiveX filter. This function has one input argument—the name of the HTTP filter on which you want to enable the ActiveX content filter.

7. Filtering Cookies

When a web client contacts a web server, the first response from the server has a **SET** cookie field in the packet header. Subsequent packets from the web client back to the web server may contain cookie information in the header in the following form:

```
"Cookie: cookie_data"
```

The web client may also have cookie information set by JavaScript code running on the web client. The Wind River Firewall cookie filter blocks cookie information from being returned to a web server by overwriting the cookie data with the same length of junk data in the message that the web client sends back to the web server, after adjusting the TCP checksum.

Use ipfirewall_http_insert_cookie_filter( ) to enable a cookie filter. This function has one input argument—the name of the HTTP filter on which you want to enable the cookie content filter.

8. Program Example

The code to add an HTTP filter and enable individual content filters is similar to the following example:

```
ipfirewall_http_add_filter("http_test");
ipfirewall_http_insert_url_filter("http_test", "www.somewebsite.com", FALSE);

ipfirewall_http_add_filter("http_key");
ipfirewall_http_insert_url_filter("http_key", "bad", TRUE);

ipfirewall_http_add_filter("http_proxy");
ipfirewall_http_insert_proxy_filter("http_proxy");

ipfirewall_http_add_filter("http_java");
ipfirewall_http_insert_java_filter("http_java");

ipfirewall_http_add_filter("http_activex");
ipfirewall_http_insert_activex_filter("http_activex");

ipfirewall_http_add_filter("http_cookie");
ipfirewall_http_insert_cookie_filter("http_cookie");
```

The preceding example is part of the firewall code and is included if the compile time macro **IPFIREWALL_USE_HTTP_FILTER_TEST** is defined.

10. KEYWORDS REFERENCE

1. IP Filter Rule Synopsis

```
[@index] {pass | block} [return-rst | return-icmp[-as-dest](return_value)] {in | out}

[log [first]] [limit [!] limit_value/unit burst [burst_value] [quick] [on [!] interface[+]]
[phys interface[+]] [pktflags pktflags_value[/mask]] [tos [tos_value][/mask]] [ttl ttl_value]
[proto [proto_value]] address_scope [icmp-type icmp_value] [flags flags_value[/mask]] [with [no]

{frag|ipopts}] [keep state [limited]| keep ifstate | keep ipsecstate]

[head | group value] [userdef id [paramstring]]
```

IP Filter Address Scope

```
{all | from {[!] any|me|ip_address[/mask]} [port {op} {port_value}] to {[!] any|me|ip_address[/mask]} [p
ort {op} {port_value}]}}
```

2. MAC Filter Rule Synopsis

```
[@index] {pass | block {in | out} [log [first]] [limit [!] limit_value/unit

burst {burst_value] [quick] [on [!] [phys interface[+]] address_scope [mac-type mac_type_value]
[802_2] [802_2-dsap [!] value] [802_2-ssap [!] value] [802_2-snap-vendor [!] value]

[802_2-snap-type [!] value] [head | group value] [userdef id [paramstring]]
```

MAC Filter Address Scope

```
{all | from {[!] any|me|mac_address[/mask]} to {[!] any|me|mac_address[/mask]}}
```

3. Keywords

! on page 55
802_2 on page 55
802_2-dsap on page 56
802_2-snap-type on page 56
802_2-snap-vendor on page 56
802_2-ssap on page 57
on page 57
all on page 57
any on page 57
block on page 58
burst on page 58

WNDRVR

first on page 58
flags on page 59
frag on page 59
from on page 60
group on page 60
head on page 61
icmp-type on page 61
in on page 61
ipopts on page 62
keep ifstate on page 62
keep ipsecstate on page 62
keep state on page 63
limit on page 63
log on page 64
mac-type on page 64
me on page 64
on on page 65
out on page 65
no on page 65
pass on page 66
phys on page 66
pktflags on page 66
port on page 67
proto on page 68
quick on page 68
return-icmp on page 68
return-icmp-as-dest on page 69
return-rst on page 71
to on page 71
tos on page 71
ttl on page 72
with on page 72

3.1. !

Description

Inverts a parameter.

Synopsis

```
keyword ! parameter
```

Parent topic: Keywords on page 54

3.2. 802_2

Description

Filters all LLC frames.

Synopsis

```
{block | pass} {in | out} [address_scope] 802_2
```

address_scope can be a unique MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.3. 802_2-dsap

Description

Filters MAC frames by the DSAP value of the LLC header.

Synopsis

```
{block | pass} {in | out} [address_scope] 802_2-dsap dsap_value
```

address_scope can be a unique MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.4. 802_2-snap-type

Description

Filters MAC frames by the DSAP value of the LLC header.

Synopsis

```
{block | pass} {in | out} [address_scope] 802_2-snap-type snap-type_value
```

address_scope can be a unique MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.5. 802_2-snap-vendor

Description

Filters MAC frames by the SNAP-type value of the LLC header.

Synopsis

```
{block | pass} {in | out} [address_scope] 802_2-snap-type snap-type_value
```

address_scope can be a unique MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**

Parent topic: Keywords on page 54

3.6. 802_2-ssap

Description

Filters MAC frames on the SSAP value of the LLC header.

Synopsis

```
{block | pass} {in | out} [address_scope] 802_2-ssap ssap_value
```

Parent topic: Keywords on page 54

3.7.

Description

Precedes a comment.

Synopsis

```
# comment
```

Parent topic: Keywords on page 54

3.8. all

Description

Specifies all traffic—that is, packets originating from any source and addressed to any destination.

Synopsis

```
{block | pass} {in | out} all
```

Parent topic: Keywords on page 54

3.9. any

Description

Specifies packets arriving from any source (with **from** keyword) or addressed to any destination (with **to** keyword).

Synopsis

```
{block | pass} {in | out} {to | from} any
```

Parent topic: Keywords on page 54

3.10. block

Description

Blocks the specified packet.

Synopsis

`block {in | out} {to | from}` *address_scope*

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.11. burst

Description

Specifies an absolute number of packets to be blocked or passed under the criteria specified by the rule. Used in conjunction with **limit**.

Synopsis

`{block | pass} {in | out}` *limit limit_value /unit* **`burst`** *burst_value* **`{to | from}`** *address_scope*

limit_value is the maximum number of packets to be accepted within the specified period of time (*unit*).

unit is second (**s**), minute (**m**), hour (**h**), or day (**d**).

burst_value is the absolute number of packets to be accepted under the criteria specified by the rule.

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me** or **any**.

Parent topic: Keywords on page 54

3.12. first

Description

Instructs the firewall to log only the first packet matching the rule. Because only a limited number of packets (1,000 by default) fits in the log, use this parameter to avoid filling up the log too fast.

Synopsis

```
{block | pass} {in | out} log [first] address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.13. flags

Description

Instructs the firewall to match TCP flags in the packet header against the specified type.

Synopsis

```
{block | pass} {in | out} proto tcp address_scope flags [flag_type [/flag_mask]]
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

flag_type and *flag_mask* can be:

- U (Urgent)
- A (Ack)
- P (Push)
- R (Reset)
- S (Syn)
- F (Fin)
- 0 (no flags active)

flag_type and *flag_mask* are separated by a slash (**/**). For a rule to match, the flag specified in *flag_type* must be set in the TCP packet header. If a flag is not explicitly specified in *flag_type*, it must not be set in the TCP packet header.

A *flag_mask*, however, introduces flexibility to the rule. Flags specified in the *flag_mask* must strictly conform to their *flag_type* setting. Flags not specified in the flag_mask are allowed to vary from their *flag_type* setting.

If no *flag_mask* is specified, all flags must match their *flag_type* specification—that is, the default mask is **UAPRSF**.

Parent topic: Keywords on page 54

3.14. frag

Description

Used to filter IP fragments (for both IPv4 and IPv6 rules).

The fragments filter keeps track of packets that are fragmented, reassembles the fragments, and then applies the firewall rules.

Synopsis

```
{block | pass} {in | out} [all] with [no] frag {from | to} address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.15. from

Description

Precedes a source address or range of addresses.

Synopsis

```
{block | pass} {in | out} from address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.16. group

Description

Identifies the group to which a rule belongs.

Arranging rules in groups can improve performance for complex rule sets. Grouping allows you to arrange rules in a treelike structure instead of a linear list. The advantage of rule grouping is that if a packet fails to match the head rule in a group, the firewall skips the remaining subrules and immediately begins matching against the next group.

When you add rules to a firewall without specifying a rule group, the new rules are added to the default group (group 0). To create a group, use the **head** keyword, followed by a group number. The number 0 is reserved for the default group. Each group must have a head rule, followed by subrules.

To assign a subrule to a group, use the **group** keyword, followed by the group number. For example:

```
block in quick on fei0 all head 1
   block in quick on fei0 from 10.0.0.0/8 to any group 1
   pass in on fei0 all group 1
block in quick on fei1 all head 2
   block in quick on fei1 from 10.0.0.0/8 to any group 2
   pass in on fei1 all group 2
pass in all
```

See also the **head** keyword (head on page 61).

WNDRVR

Synopsis

```
{block | pass} {in | out} address_scope group group_number
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

group_number is the number of the rule group to which the rule belongs.

Parent topic: Keywords on page 54

3.17. head

Description

Identifies the head rule of a group.

See also the **group** keyword (group on page 60).

Synopsis

```
{block | pass} {in | out} address_scope head head_number
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

head_number is the number of the group for which this rule is the head rule.

Parent topic: Keywords on page 54

3.18. icmp-type

Description

In an IP filter rule, specifies the ICMP type.

Synopsis

```
{block | pass} {in | out} {to | from} address_scope proto icmp [icmp-type type_value]
[code code_value]
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

type_value is the ICMP type field in the IP packet header.

code_value is the ICMP code field for the specified ICMP type.

Parent topic: Keywords on page 54

3.19. in

Description

Indicates an incoming packet.

Synopsis

```
{block | pass} in address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.20. ipopts

Description

Used to filter IP options (for IPv4 rules only).

Synopsis

```
{block | pass} {in | out} [all] with [no] ipopts {from | to} address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.21. keep ifstate

Description

Enables stateful firewalling by temporarily opening a port for incoming traffic when an outgoing packet matches the specified rule. With the **keep ifstate** keyword, the firewall tracks the state of an existing connection on the basis of source IP address, destination IP address, source port, destination port, protocol, and interface.

Keeping state focuses on blocking or passing packets from a new connection. If the new connection is allowed, all packets are passed in both directions. If a new connection is blocked, all packets are blocked in both directions.

Synopsis

```
{block | pass} {in | out} {to | from} address_scope keep ifstate
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.22. keep ipsecstate

Description

Enables stateful firewalling by temporarily opening a port for incoming traffic when an outgoing packet matches the specified rule. With the **keep ipsecstate** keyword, the firewall tracks the state of a connection based on the use of encryption by IPsec. For outgoing packets, if encryption is in use, the VPN gateway address is stored in the firewall state entry. For incoming packets, the packet must be issued from the same VPN gateway before it is forwarded to the destination.

Keeping state focuses on blocking or passing packets from a new connection. If the new connection is allowed, all packets are passed in both directions. If a new connection is blocked, all packets are blocked in both directions.

Synopsis

```
{block | pass} {in | out} {to | from} address_scope keep ipsecstate
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.23. keep state

Description

Enables stateful firewalling by temporarily opening a port for incoming traffic when an outgoing packet matches the specified rule. With the **keep state** keyword, the firewall tracks the state of an existing connection on the basis of source IP address, destination IP address, source port, destination port, and protocol.

Keeping state focuses on blocking or passing packets from a new connection. If the new connection is allowed, all packets are passed in both directions. If a new connection is blocked, all packets are blocked in both directions.

Synopsis

```
{block | pass} {in | out} {to | from} address_scope keep state
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.24. limit

Description

Specifies the number of packets to be accepted within a given time frame under the criteria specified by the rule. Used in conjunction with **burst**.

Synopsis

```
{block | pass} {in | out} limit limit_value /unit burst burst_value {to | from} address_scope
```

limit_value is the maximum number of packets to be accepted within the specified period of time (**unit**).

WNDRVR

unit is second (**s**), minute (**m**), hour (**h**), or day (**d**).

burst_value is the absolute number of packets to be accepted under the criteria specified by the rule.

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.25. log

Description

Instructs the firewall to log packets matching the rule.

Synopsis

```
{block | pass} {in | out} log [first] address_scope
```

first instructs the firewall to log only the first packet matching the rule. Because only a limited number of packets (1,000 by default) fits in the log, use this parameter to avoid filling up the log too fast.

address_scope can be a unique IP address, an address space; or the keywords *!*, *all*, **me**, or **any**.

Parent topic: Keywords on page 54

3.26. mac-type

Description

In a MAC filter rule, specifies the MAC frame type.

Synopsis

```
{block | pass} {in | out} {to | from} address_scope mac-type mac_type_value
```

address_scope can be a unique MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.27. me

Description

In an IP filter rule, specifies any address configured on the system.

In a MAC filter rule, specifies the MAC address assigned to the interface the packet is sent or received on.

Synopsis

```
{block | pass} {in | out} me
```

Parent topic: Keywords on page 54

3.28. on

Description

Precedes an interface specification.

Synopsis

```
{block | pass} {in | out} on interface [+] address_scope
```

interface is an interface name.

The plus sign (**+**) is used as a wildcard to specify any character or digit in an interface name.

address_scope can be a unique IP or MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.29. out

Description

Indicates an outgoing packet.

Synopsis

```
{block | pass} out address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.30. no

Description

Inverts the IP fragments or IP options setting specified in a **with frag** or **with ipopts** rule.

Synopsis

```
{block | pass} {in| out} {to | from} address_scope with no {frag | ipopts}
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.31. pass

Description

Accepts the specified packet.

Synopsis

```
pass {in | out} {to | from} address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.32. phys

Description

Specifies a physical interface (as opposed to a virtual driver, or mirror interface) to be used in a filter rule.

To enable this option, see Source Build Configuration Options on page 6.

Synopsis

```
{block | pass} {in | out} on interface [+] phys interface [+] address_scope
```

interface is an interface name.

The plus sign (**+**) is used as a wildcard to specify any character or digit in an interface name.

address_scope can be a unique IP or MAC address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.33. pktflags

Description

Specifies a packet flag to indicate prior processing by IPsec, NAT, or tunneling.

Synopsis

```
{block | pass} in pktflags [S | N | T | O] [/ [S | N | T ]
```

Parent topic: Keywords on page 54

3.34. port

Description

Specifies a port for a UDP or TCP packet.

Synopsis

{block | pass} {in | out} proto *proto_value* **{to | from}** *address_scope* **port** *op port_value*

proto_value is **tcp** or **udp**.

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

op is a mathematical operator. Wind River Firewall recognizes the operators defined in Operators Valid with Port Keyword on page 67, which can be specified using mathematical signs or text.

Table 1. Operators Valid with Port Keyword

Operator	Text Designation	Description
"="	eq	equal
"!="	ne	not equal
"<"	lt	less than
"<="	le	less than or equal
">"	gt	greater than
">="	ge	greater than or equal to
"<>"	or	outside range
"><"	ir	inside range

port *port_value* **op** *port_value*

70 For "**<>**" and "**><**", the syntax is actually as follows:

For example, a specification such as **port 10000 "<>" 20000** means that all port numbers less than 10000 or greater than 20000 match the rule.

port_value is an individual port or an interval.

Parent topic: Keywords on page 54

3.35. proto

Description

Specifies an Internet protocol.

Synopsis

```
{block | pass} {in | out} proto proto_value address_scope [port op port_value]
```

proto_value is **tcp** or **udp**.

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

op is a mathematical operator. For more information, see the *Operators Valid with Port Keyword on page 67* table.

port_value is an individual port or an interval.

Parent topic: Keywords on page 54

3.36. quick

Description

Instructs the firewall to abort processing and immediately take the action specified in the rule.

Synopsis

```
{block | pass} {in | out} quick address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.37. return-icmp

Description

Sends a destination unreachable error back to a peer if an ICMP packet specified by the rule is blocked by the firewall.

Synopsis

```
block in return-icmp[(number)] [proto udp] {from | to} address_scope [port op port_value]
```


WNDRVR

number indicates the ICMP destination unreachable code field to be set in the response message. It can be any value from 0-255. If no value is supplied, 0 is assumed. Return-ICMP Codes on page 69 defines the supported options.

Table 1. Return-ICMP Codes

Code	Description
IPv4 Codes	
0	Network unreachable
1	Host unreachable
2	Protocol unreachable
3	Port unreachable
4	Fragmentation needed but no frag bit set
5	Source routing failed
9	Destination network administratively prohibited
10	Destination host administratively prohibited
IPv6 Codes	
0	Destination unreachable: no route
2	Destination unreachable: beyond scope
3	Destination unreachable: addr
4	Destination unreachable: no port

proto_value can be any protocol.

address_scope can be a unique IP address, an address space, or the keywords **me** or **any**.

op is a mathematical operator. For more information, see the Operators Valid with Port Keyword on page 67 table.

port_value is an individual port or an interval.

Parent topic: Keywords on page 54

3.38. return-icmp-as-dest

Description

Sends a destination unreachable error back to a peer if an ICMP packet specified by the rule is blocked by the firewall. When this keyword is used, the destination unreachable error contains a source address copied from the destination address of the blocked packet.

Synopsis

```
block in return-icmp-as-dest[(number)] [proto udp] {from | to} address_scope [port op
port_value]
```

number indicates the ICMP destination unreachable code field to be set in the response message. It can be any value from 0-255. If no value is supplied, 0 is assumed. The Return-ICMP-as-Dest Codes on page 70 table defines the supported options.

Table 1. Return-ICMP-as-Dest Codes

Code	Description
IPv4 Codes	
0	Network unreachable
1	Host Unreachable
2	Protocol unreachable
3	Port unreachable
4	Fragmentation needed but no frag bit set
5	Source routing failed
9	Destination network administratively prohibited
10	Destination host administratively prohibited
IPv6 Codes	
0	Destination unreachable: no route
2	Destination unreachable: beyond scope
3	Destination unreachable: addr
4	Destination unreachable: no port

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

op is a mathematical operator. For more information, see the Operators Valid with Port Keyword on page 67 table.

port_value is an individual port or an interval.

Parent topic: Keywords on page 54

3.39. return-rst

Description

Sends a reset segment (connection refused error) back to a peer if a TCP packet specified by the rule is blocked by the firewall.

Synopsis

```
block in return-rst proto tcp address_scope [port op port_value]
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

op is a mathematical operator. For more information, see the Operators Valid with Port Keyword on page 67 table.

port_value is an individual port or an interval.

Parent topic: Keywords on page 54

3.40. to

Description

Precedes a destination address or range of addresses.

Synopsis

```
{block | pass} {in | out} to address_scope
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.41. tos

Description

Specifies a value in the type of service (**tos**) field of an IPv4 packet header or the quality class field of an IPv6 packet header.

Synopsis

```
{block | pass} {in| out} tos tos_value[/tos_mask] address_scope
```

tos_value is the TOS value field in an IPv4 packet header or the quality class field in an IPv6 packet header. It must be specified in hexadecimal. The leading 0 or 0x is optional.

tos_mask is a full mask, which is bitwise combined with the tos field in the protocol header using a boolean **AND**, then compared with the *tos_value* in the rule. It must be specified in hexadecimal. The leading 0 or 0x is optional.

Parent topic: Keywords on page 54

3.42. ttl

Description

Specifies a value in the time to live (**ttl**) field of an IP packet header. This value specifies a timeout for received fragments.

Synopsis

```
{block | pass} {in| out} ttl ttl_value address_scope
```

ttl_value ranges from 0-255.

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

3.43. with

Description

Enables a rule to match packets containing IP options and fragments. Use with **ipopts** for IPv4 rules and with **frag** for IPv4 or IPv6 rules. It is also possible to insert a **no** keyword to match packets not including fragments or IP options.

Synopsis

```
{block | pass} {in| out} {to | from} address_scope with [no] {frag | ipopts}
```

address_scope can be a unique IP address, an address space; or the keywords **!**, **all**, **me**, or **any**.

Parent topic: Keywords on page 54

11. SHELL COMMAND REFERENCE

1. ipf

Name

ipf – enable, disable, and modify the firewall.

Synopsis

```
ipf [rule]{[-6 rule][-f filename][-r rule][-n rule][-m rule]
[-D][-E][-F {[r][s][l][u][a][gid]}][-P {[r][s][l][u][a][gid]}][-S][-V][-Z]}
```

Description

ipf configures IP filter and MAC filter rules. The command runs on a target shell. The default operation is to add an IP filter rule. Use the -m flag to add a MAC filter rule and the -r flag to remove rules. ipf can also be used to display or flush rule, state, or log tables.

> Note: To run this command, you must switch to the command interpreter shell before running the ipf command. Type cmd at the command prompt. Then run the ipf command.

The shell command options are as follows:

-m

Add a MAC filter rule.

-6

Add an IPv6 filter rule.

-r

Remove rule.

-f

Add rules from the specified file.

-n

Check rule syntax.

-D

Disable the firewall.

-E

Enable the firewall.

-F{[r][s][l][u][a][g*id*]}

Flush table(s). To flush a particular table, use the following options, either singly or in combination:

r (rules)

s (state)

l (log)

u (user)

a (all)

gid (rule group, where id specifies the group number)

-P{[r][s][l][u][a][g*id*]}

Display table(s). To display a particular table, use the following options, either singly or in combination:

r (rules)

s (state)

l (log)

u (user)

a (all)

gid (rule group, where id specifies the group number)

-S

Display statistics.

-V

Show firewall version.

-Z

Clear statistics.